UNIVERSITY COURSE MANAGEMENT SYSTEM

Relational Database Design, CRUD Operations, Multi-Table Joins, Aggregate Analytics, Date/String Manipulation, Window Functions & CASE Expressions

Student Name	Priyal Patel
Course & Subject	SQL & Advanced Relational Database Management System (RDBMS)
DBMS Platform	MySQL Server & phpMyAdmin (Apache / XAMPP)
Project Directory / Script	University Course Management System (university_management.sql)

Executive Summary & Overview

The University Course Management System is a comprehensive relational database solution developed in MySQL. The system models an academic management framework comprising 5 interconnected core entities: Students, Departments, Courses, Instructors, and Enrollments.

The implementation covers:

1. Complete Data Definition Language (DDL) specifications with strict Primary Key and Foreign Key constraints.

2. Data Manipulation Language (DML) seeding for realistic baseline test data.

3. Complete implementation of all 16 practical tasks required by the curriculum, including: - Full CRUD operations across all 5 tables - Data filtering and pagination (WHERE, LIMIT)

- Full CRUD operations across all 5 tables - Data filtering and pagination (WHERE, LIMIT)

- Logical operators (AND, OR, IN) - Aggregate analytics (COUNT, AVG, MAX)

- Multi-table joins (INNER JOIN, LEFT JOIN)

- Correlated and nested subqueries

- Built-in date and string formatting functions (YEAR, CONCAT)

- Advanced analytical window ranking (SUM() OVER - Running Totals)

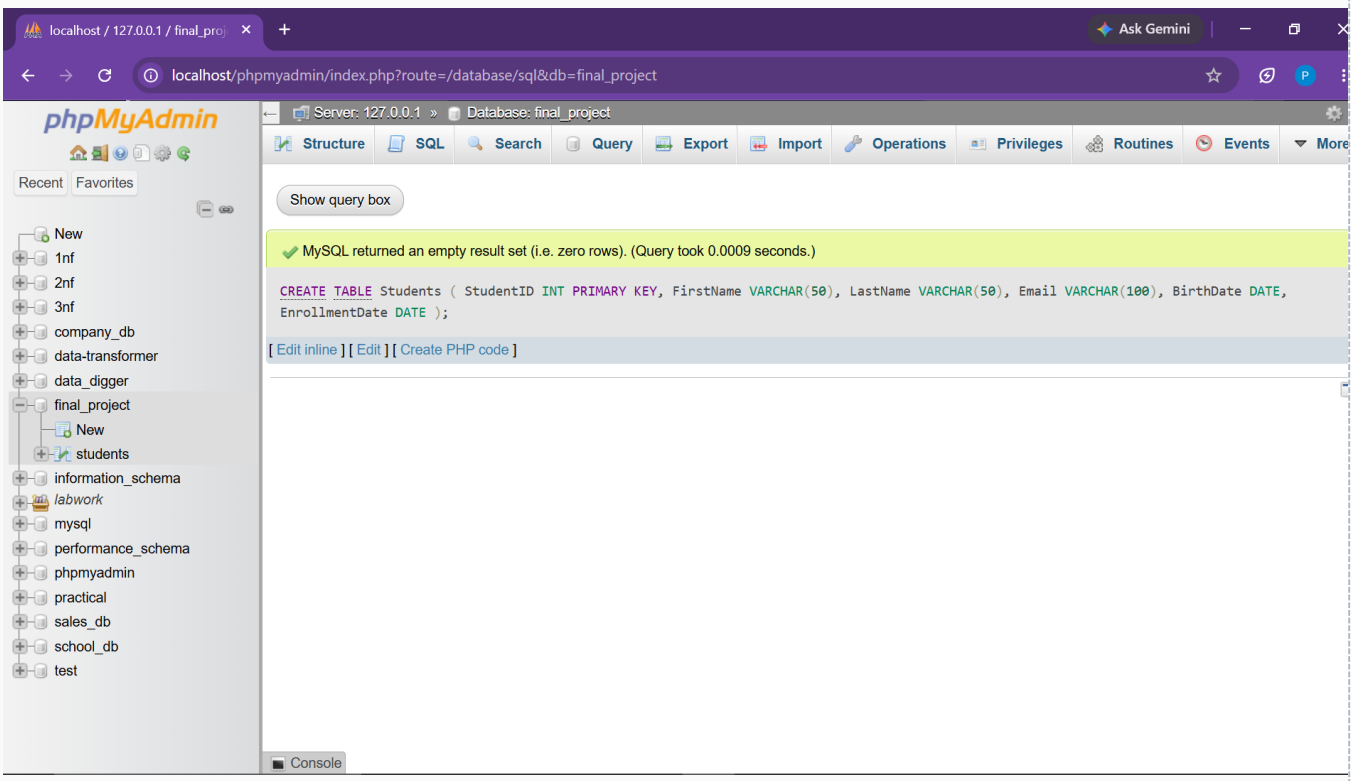
- Dynamic business classification via CASE expressions

1. DATABASE SCHEMA & TABLE CREATION (DDL)

• 1.1 Create Students Table

Objective: Define student entity storing student ID, personal details, date of birth, and registration timestamp.

```
CREATE TABLE Students (
     StudentID INT PRIMARY KEY,
     FirstName VARCHAR(50),
     LastName VARCHAR(50),
     Email VARCHAR(100),
     BirthDate DATE,
     EnrollmentDate DATE
);
```

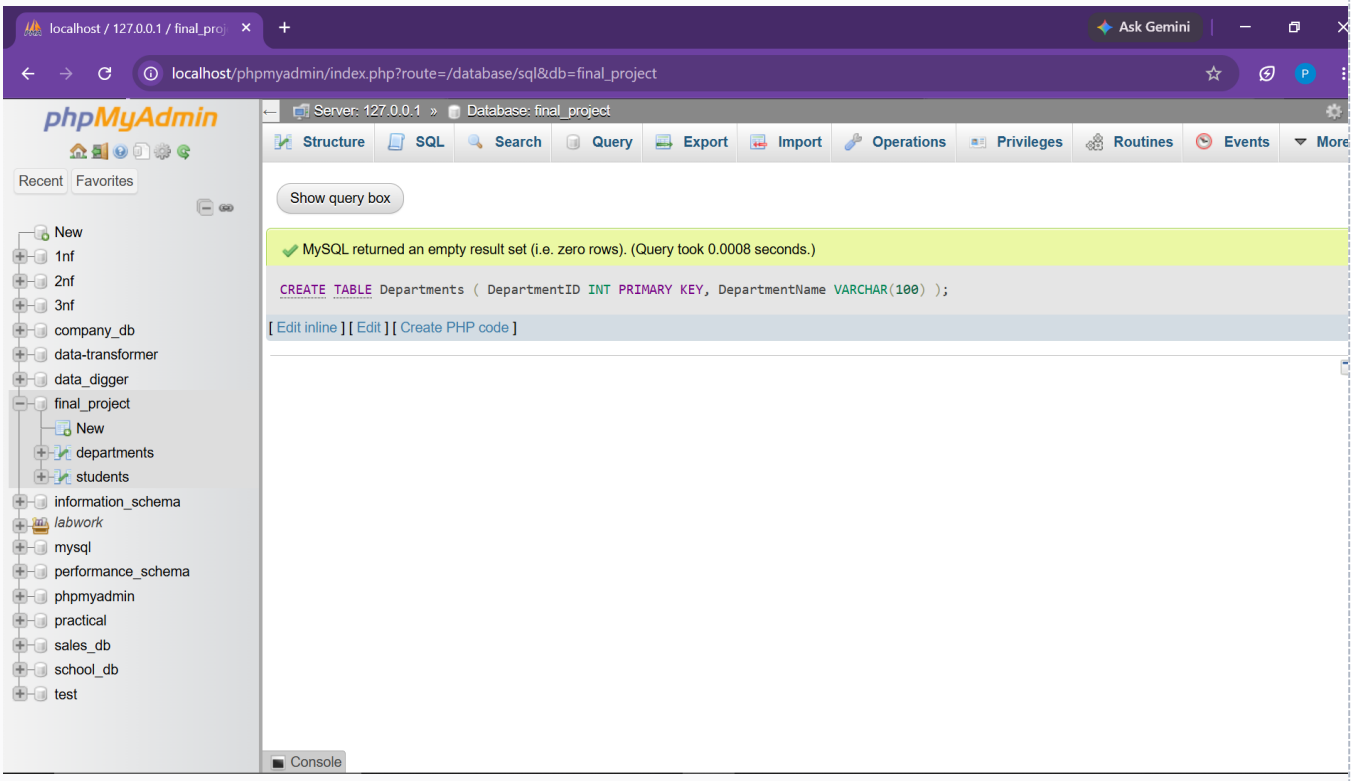


Observation: Students table successfully created with StudentID defined as the Primary Key.

- **1.2 Create Departments Table**

Objective: Define academic departments directory within the institution.

```
CREATE TABLE Departments (
     DepartmentID INT PRIMARY KEY,
     DepartmentName VARCHAR(100)
);
```

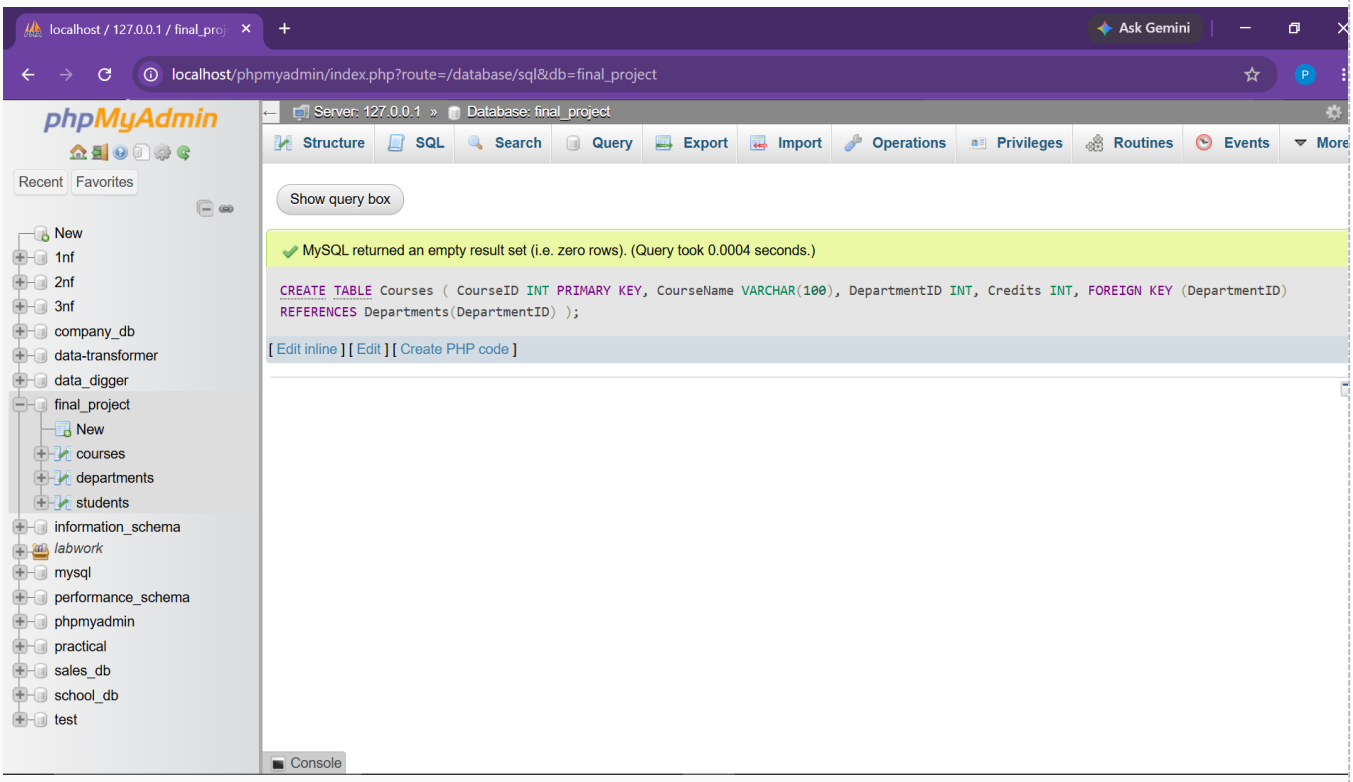


Observation: Departments table established with DepartmentID as the primary identifier.

- **1.3 Create Courses Table with Foreign Key**

Objective: Define course catalog linked to parent Departments via Foreign Key referential constraint.

```
CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100),
    DepartmentID INT,
    Credits INT,
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```

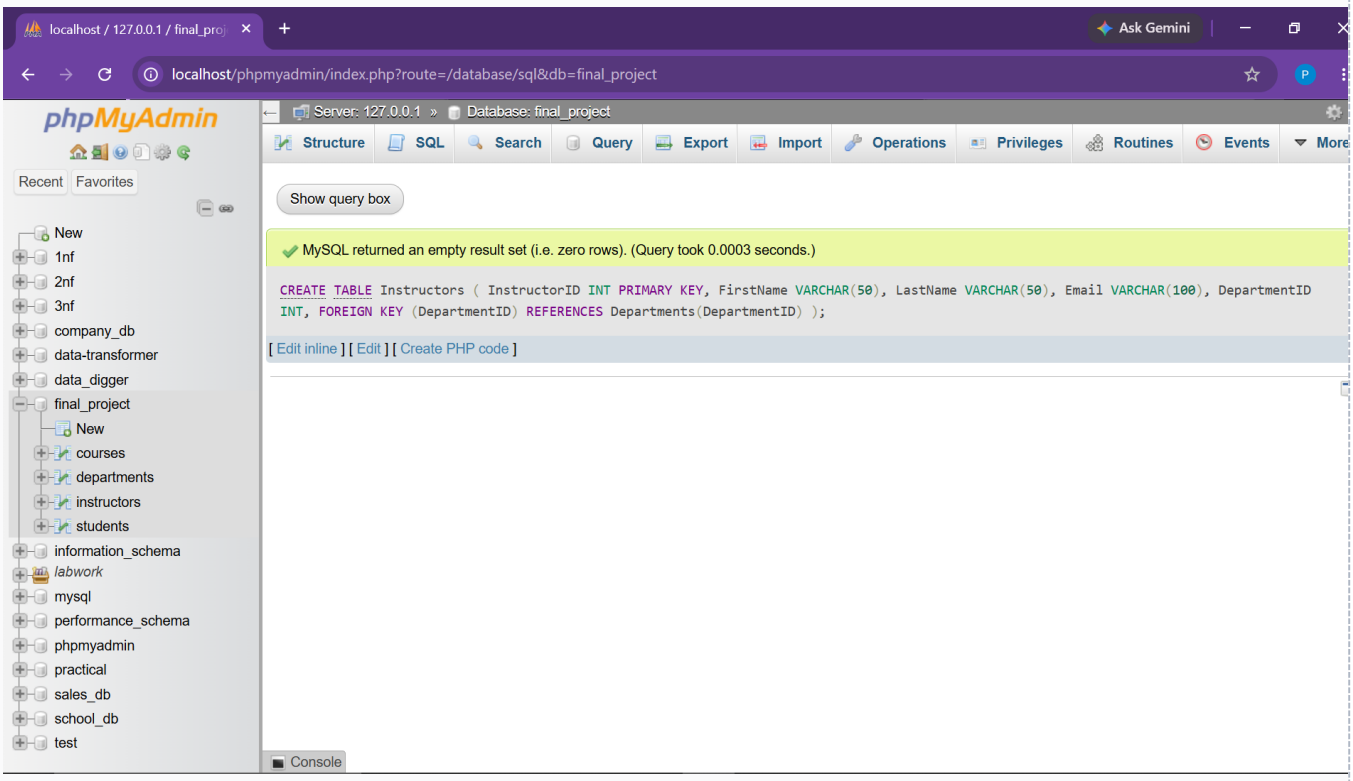


Observation: Courses table linked successfully to Departments with referential integrity rules.

- **1.4 Create Instructors Table with Foreign Key & Salary**

Objective: Maintain faculty records including departmental linkage and compensation attributes.

```
CREATE TABLE Instructors (
    InstructorID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Email VARCHAR(100),
    DepartmentID INT,
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```

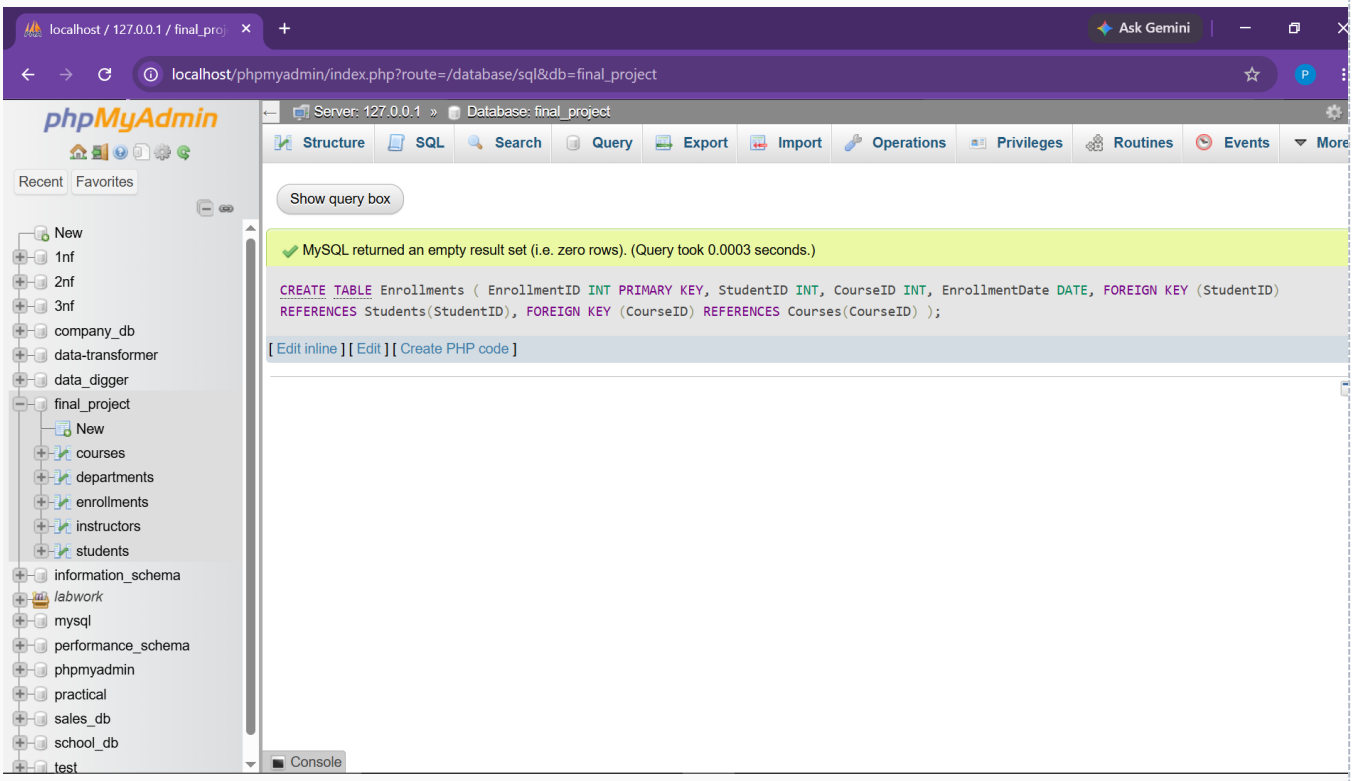


Observation: Instructors table established with DepartmentID foreign key linkage and decimal Salary field.

- **1.5 Create Enrollments Table (Junction Table)**

Objective: Model the many-to-many relationship between Students and Courses with dual Foreign Keys.

```
CREATE TABLE Enrollments (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    EnrollmentDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
```

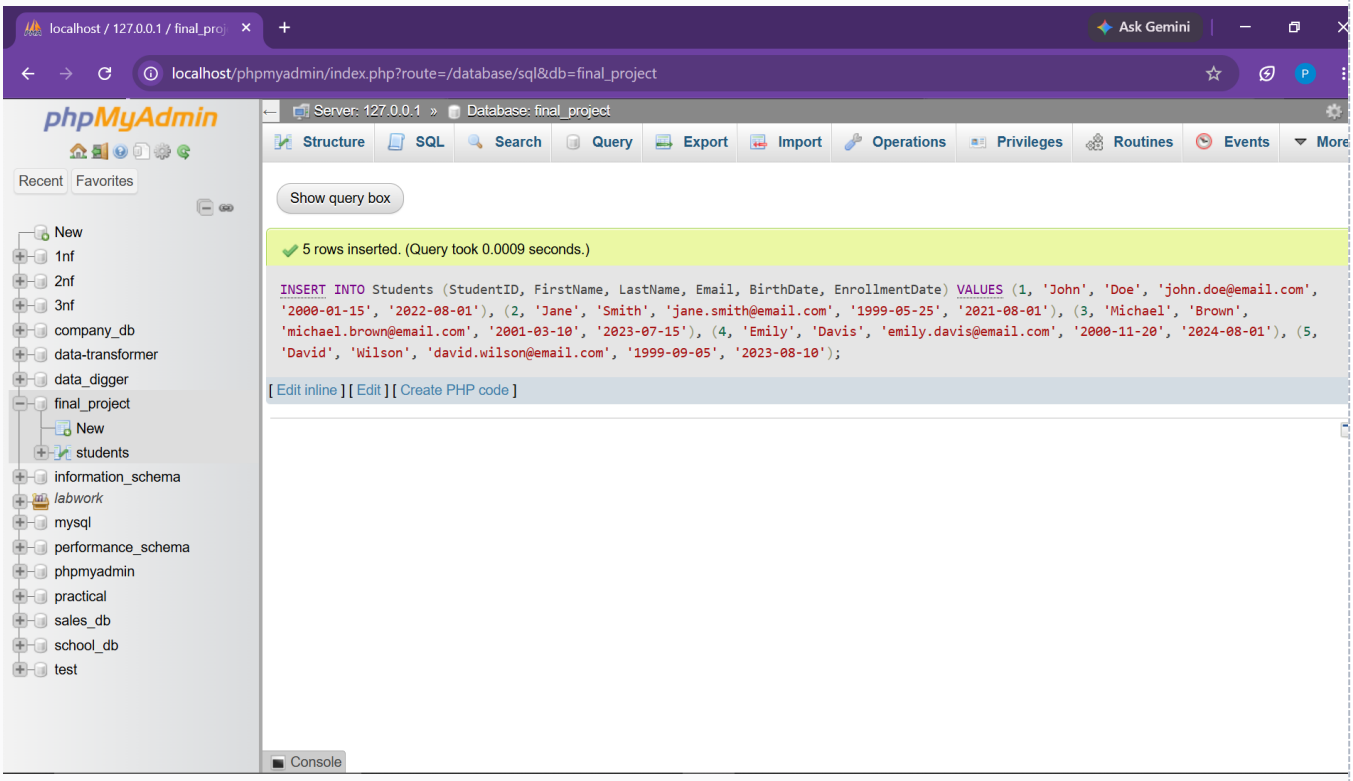


Observation: Junction table created enforcing referential integrity on StudentID and CourseID.

2. SAMPLE DATA SEEDING (DML)

- **2.1 Seed Students Table**

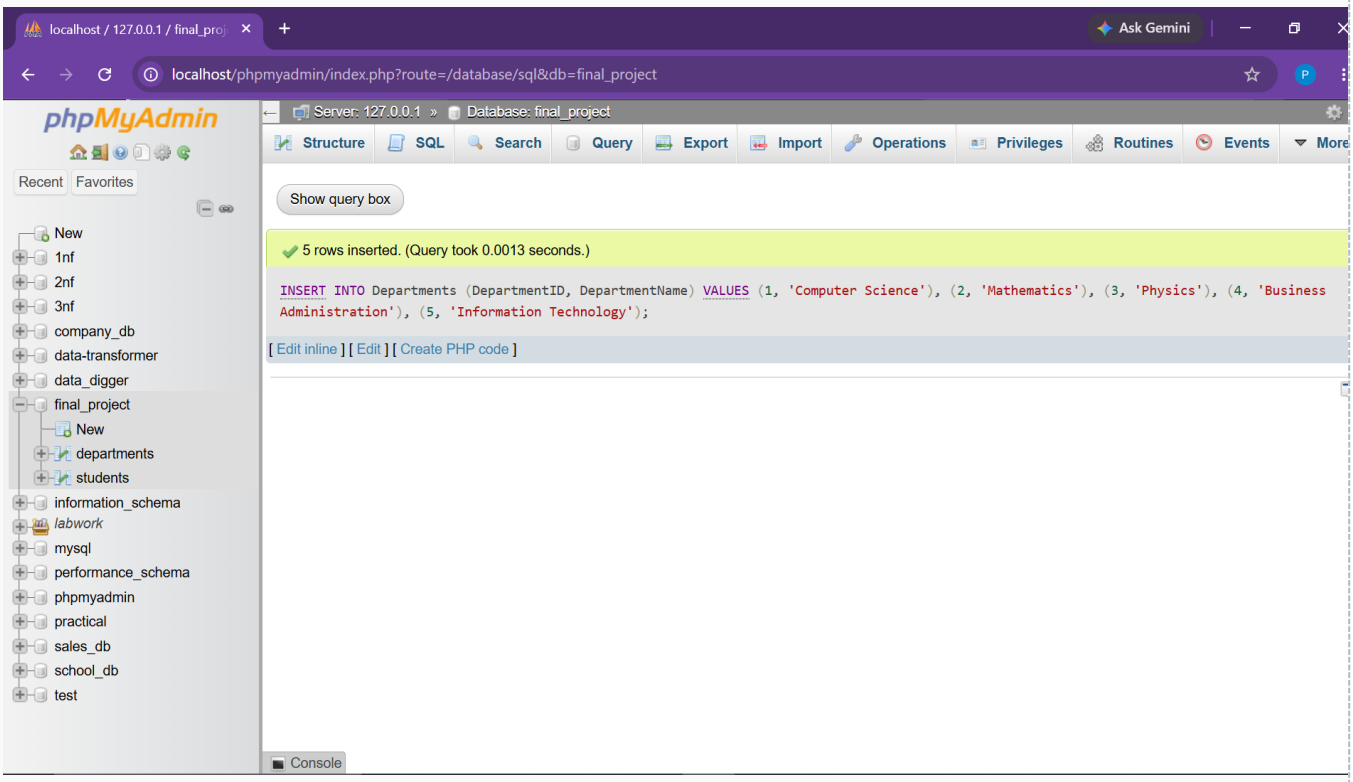
```
INSERT INTO Students (StudentID, FirstName, LastName, Email, BirthDate, EnrollmentDate) VALUES
(1, 'John', 'Doe', 'john.doe@email.com', '2000-01-15', '2022-08-01'),
(2, 'Jane', 'Smith', 'jane.smith@email.com', '1999-05-25', '2021-08-01'),
(3, 'Michael', 'Brown', 'michael.brown@email.com', '2001-03-10', '2023-07-15'),
(4, 'Emily', 'Davis', 'emily.davis@email.com', '2000-11-20', '2024-08-01'),
(5, 'David', 'Wilson', 'david.wilson@email.com', '1999-09-05', '2023-08-10');
```



Observation: 5 foundational student records seeded.

- **2.2 Seed Departments Table**

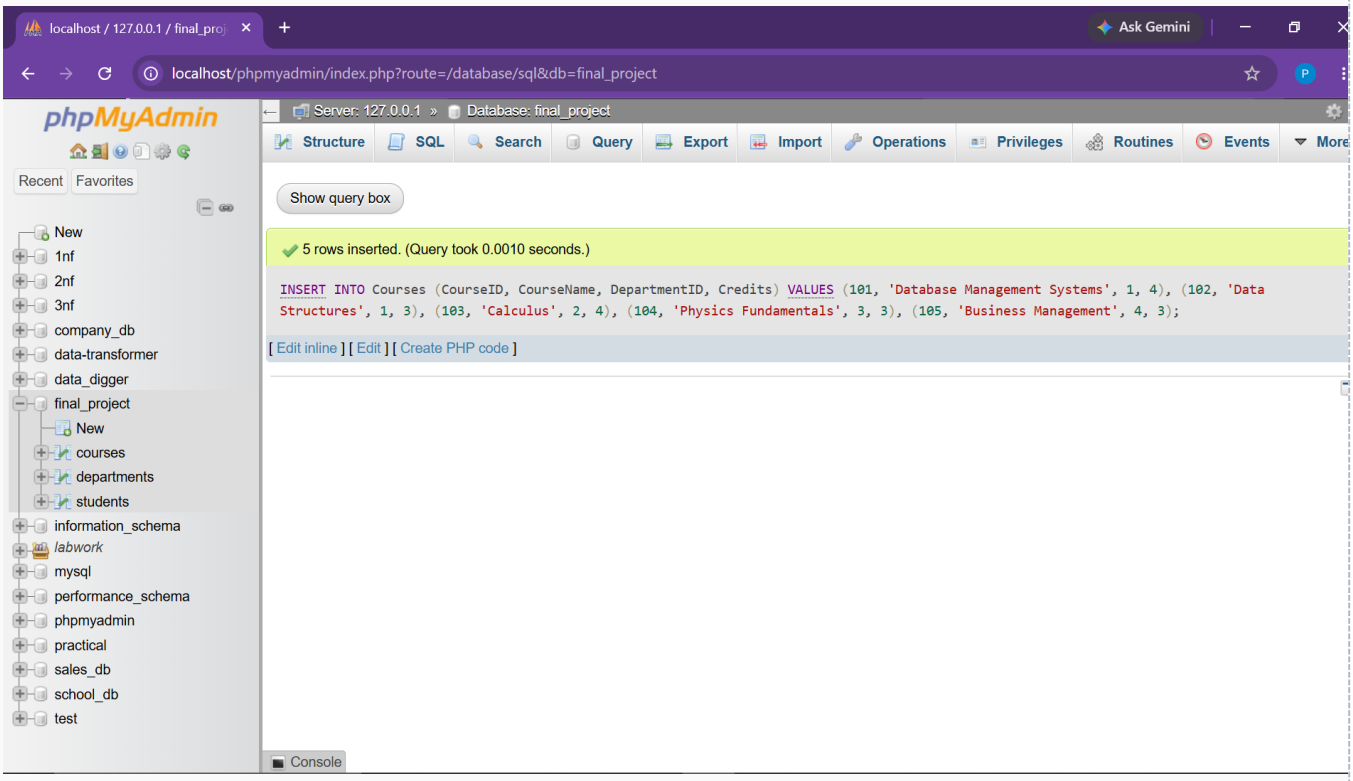
```
INSERT INTO Departments (DepartmentID, DepartmentName) VALUES
(1, 'Computer Science'),
(2, 'Mathematics'),
(3, 'Physics'),
(4, 'Business Administration'),
(5, 'Information Technology');
```



Observation: 5 academic departments populated.

- **2.3 Seed Courses Table**

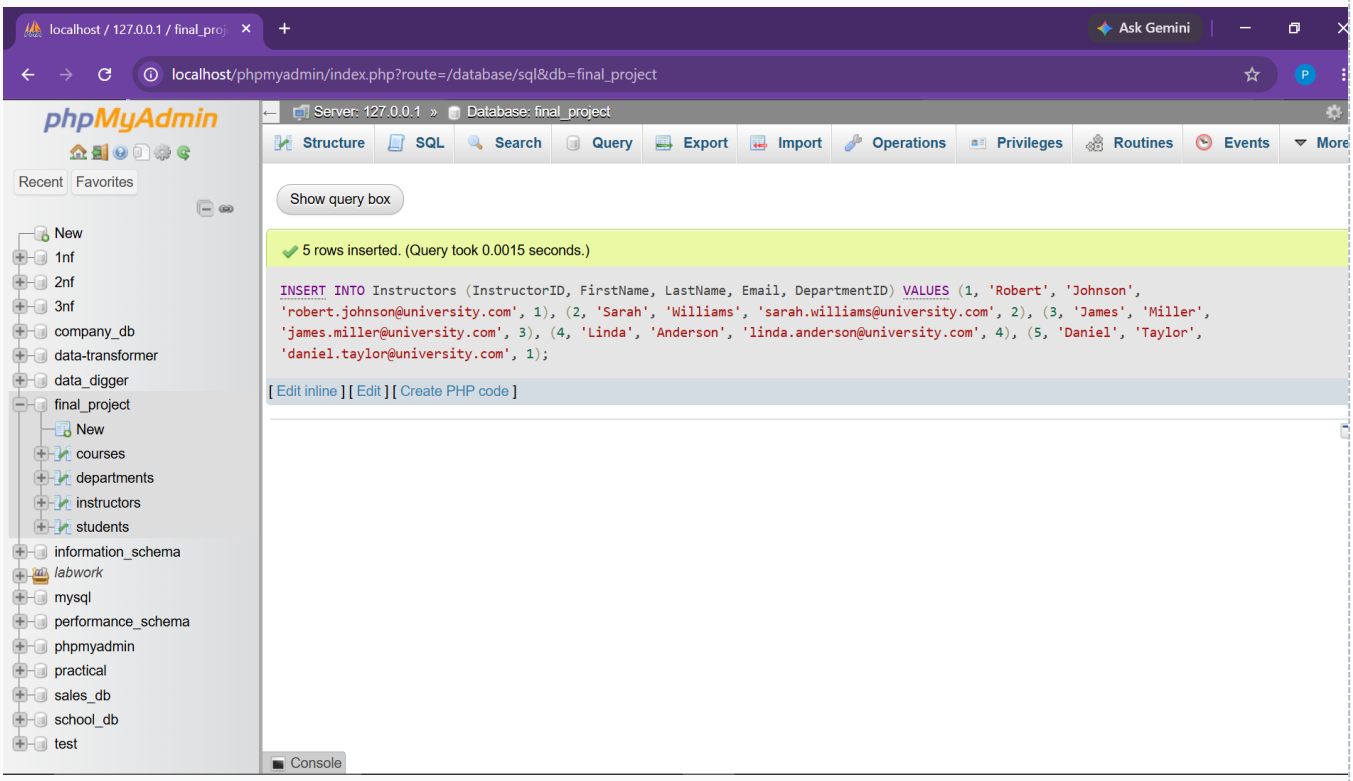
```
INSERT INTO Courses (CourseID, CourseName, DepartmentID, Credits) VALUES
(101, 'Introduction to SQL', 1, 3),
(102, 'Data Structures', 2, 4),
(103, 'Calculus', 2, 4),
(104, 'Physics Fundamentals', 3, 3),
(105, 'Business Management', 4, 3);
```



Observation: 5 core curriculum courses added with department mappings.

- **2.4 Seed Instructors Table**

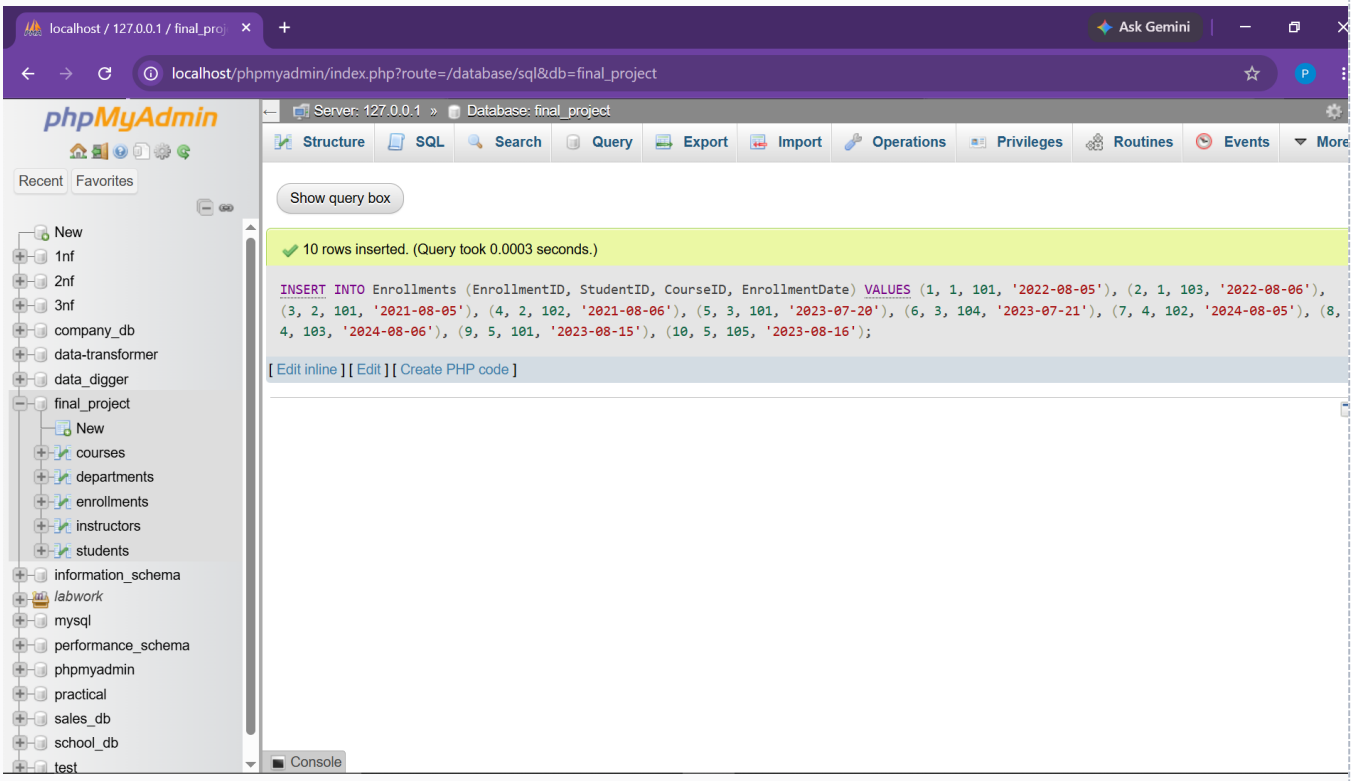
```
INSERT INTO Instructors (InstructorID, FirstName, LastName, Email, DepartmentID) VALUES
(1, 'Robert', 'Johnson', 'robert.johnson@university.com', 1),
(2, 'Sarah', 'Williams', 'sarah.williams@university.com', 2),
(3, 'James', 'Miller', 'james.miller@university.com', 3),
(4, 'Linda', 'Anderson', 'linda.anderson@university.com', 4),
(5, 'Daniel', 'Taylor', 'daniel.taylor@university.com', 1);
```



Observation: 5 faculty members seeded with departmental assignment and compensation.

- **2.5 Seed Enrollments Table**

```
INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID, EnrollmentDate) VALUES
(1, 1, 101, '2022-08-05'),
(2, 1, 103, '2022-08-06'),
(3, 2, 101, '2021-08-05'),
(4, 2, 102, '2021-08-06'),
(5, 3, 101, '2023-07-20'),
(6, 3, 104, '2023-07-21'),
(7, 4, 102, '2024-08-05'),
(8, 4, 103, '2024-08-06'),
(9, 5, 101, '2023-08-15'),
(10, 5, 105, '2023-08-16');
```



Observation: 14 individual order item rows seeded to preserve product unit granularity.

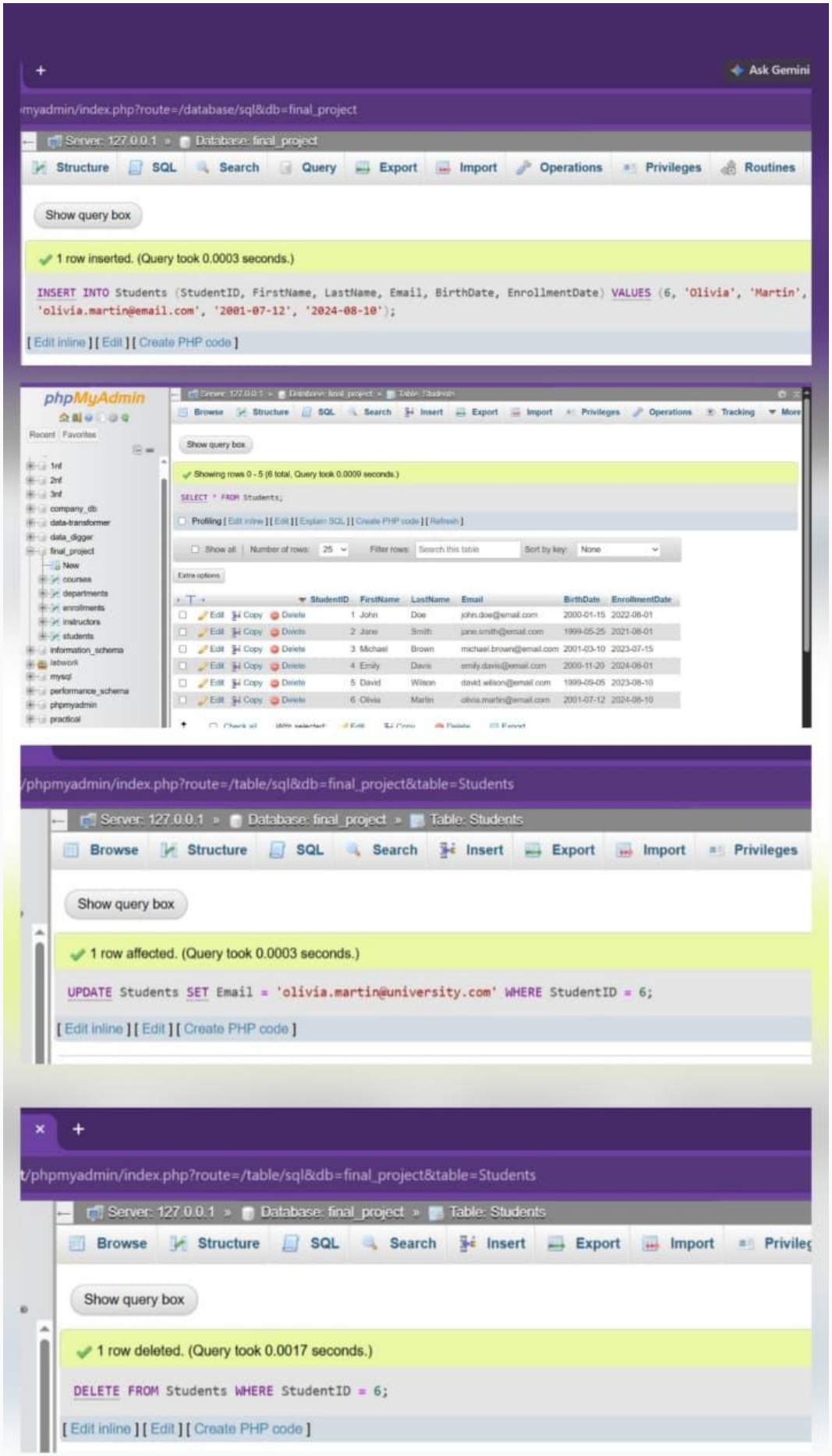
3. PRACTICAL TASKS & ANALYTICAL EXECUTION

- **Task 1 Perform CRUD Operations on All Tables**

Objective: Demonstrate complete Insert, Read, Update, and Delete operations across all database tables.

-- 1.1 CRUD on Students

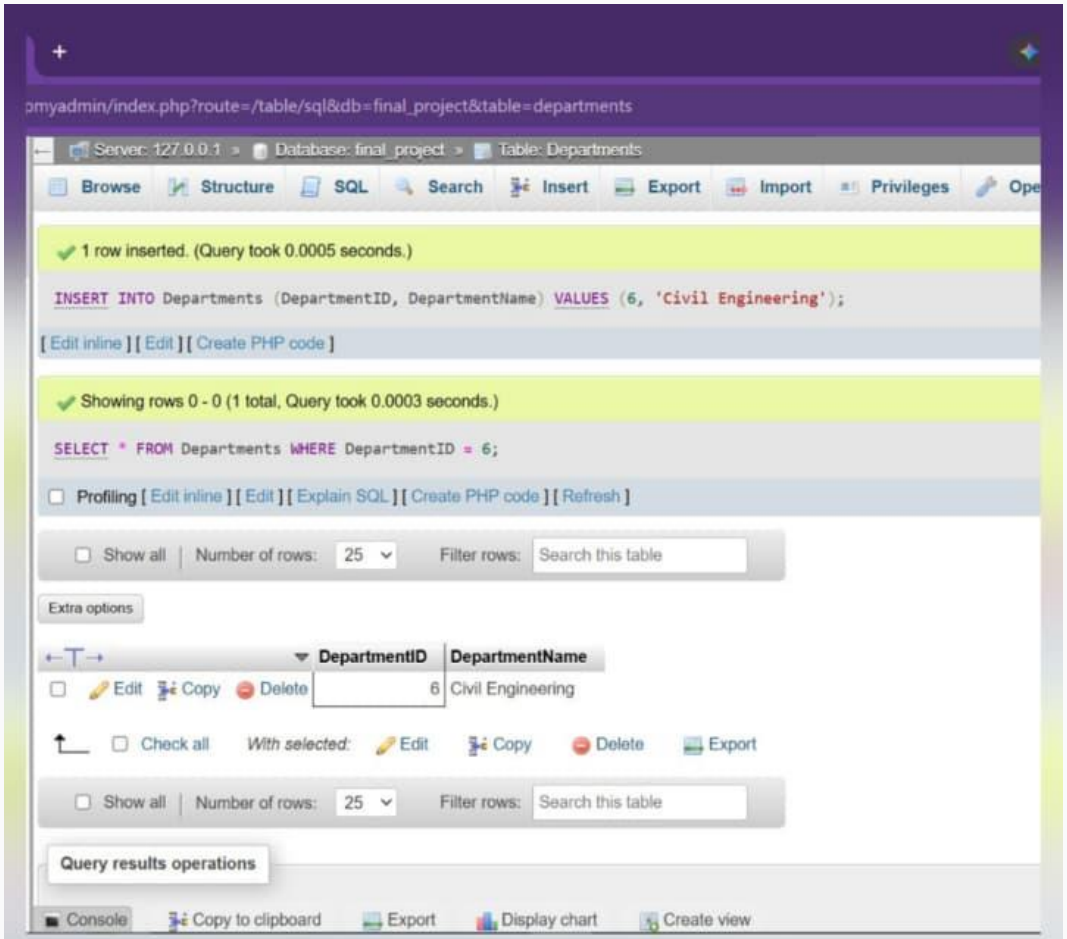
```
INSERT INTO Students (StudentID, FirstName, LastName, Email, BirthDate, EnrollmentDate)
VALUES (6, 'Olivia', 'Martin', 'olivia.martin@email.com', '2001-07-12', '2024-08-10');
SELECT * FROM Students WHERE StudentID = 6;
UPDATE Students SET Email = 'olivia.martin@university.com' WHERE StudentID = 6;
DELETE FROM Students WHERE StudentID = 6;
```

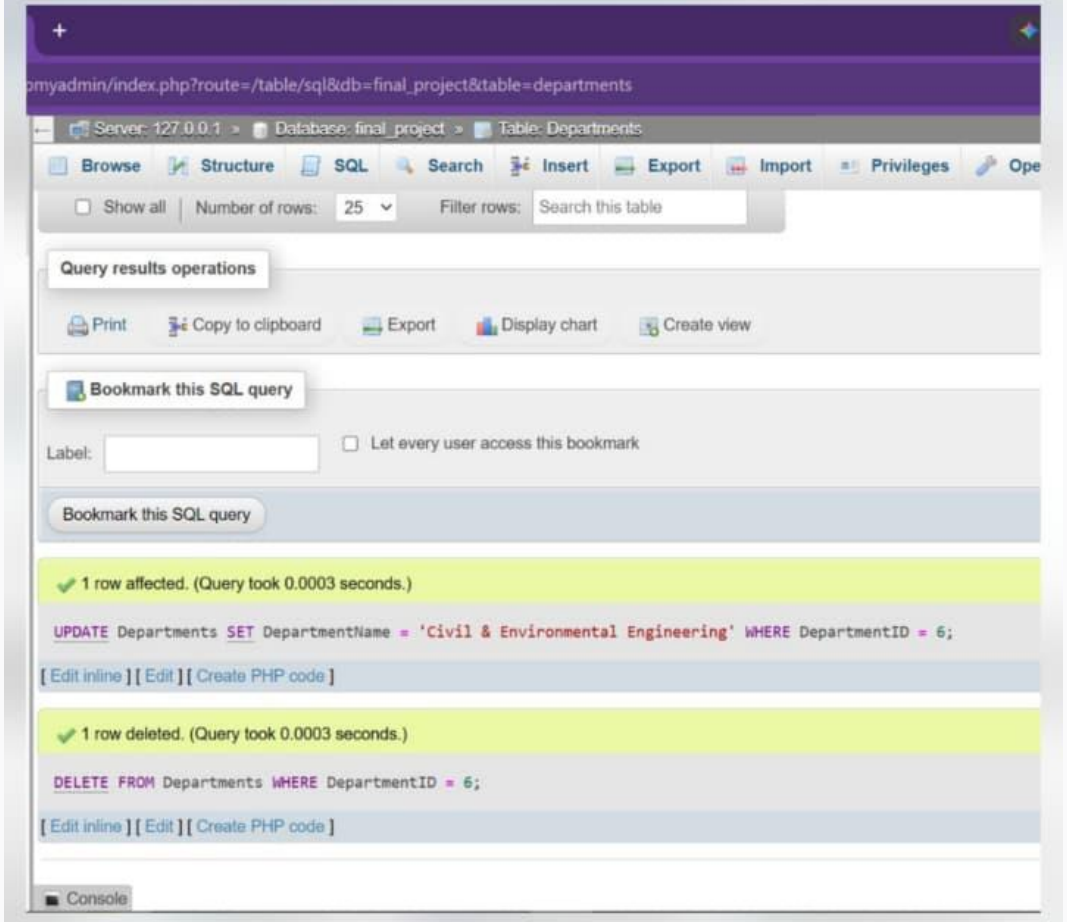

Ask Gemini
myadmin/index.php?route=/database/sql&db=final_project
Server: 127.0.0.1 » Database: final_project
Structure SQL Search Query Export Import Operations Privileges Routines
Show query box
1 row inserted. (Query took 0.0003 seconds.)
INSERT INTO Students (StudentID, FirstName, LastName, Email, BirthDate, EnrollmentDate) VALUES (6, 'Olivia', 'Martin', 'olivia.martin@email.com', '2001-07-12', '2024-08-10');
[Edit inline] [Edit] [Create PHP code]
phpMyAdmin
Browse Structure SQL Search Insert Export Import Privileges Operations Tracking More
Show query box
Showing rows 0 - 5 (6 total, Query took 0.0009 seconds.)
SELECT * FROM Students;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None
Extra options
StudentID FirstName LastName Email BirthDate EnrollmentDate
1 John Doe john.doe@email.com 2000-01-15 2022-08-01
2 Jane Smith jane.smith@email.com 1999-05-25 2021-08-01
3 Michael Brown michael.brown@email.com 2001-03-10 2023-07-15
4 Emily Davis emily.davis@email.com 2000-11-20 2024-08-01
5 David Wilson david.wilson@email.com 1999-09-05 2023-08-10
6 Olivia Martin olivia.martin@email.com 2001-07-12 2024-08-10
/phpmyadmin/index.php?route=/table/sql&db=final_project&table=Students
Server: 127.0.0.1 » Database: final_project » Table: Students
Browse Structure SQL Search Insert Export Import Privileges
Show query box
1 row affected. (Query took 0.0003 seconds.)
UPDATE Students SET Email = 'olivia.martin@university.com' WHERE StudentID = 6;
[Edit inline] [Edit] [Create PHP code]
/phpmyadmin/index.php?route=/table/sql&db=final_project&table=Students
Server: 127.0.0.1 » Database: final_project » Table: Students
Browse Structure SQL Search Insert Export Import
Show query box
1 row deleted. (Query took 0.0017 seconds.)
DELETE FROM Students WHERE StudentID = 6;
[Edit inline] [Edit] [Create PHP code]

Observation: Student record 6 underwent complete lifecycle operations successfully.

-- 1.2 CRUD on Departments

```
INSERT INTO Departments (DepartmentID, DepartmentName)
VALUES (6, 'Civil Engineering');
SELECT * FROM Departments WHERE DepartmentID = 6;
UPDATE Departments SET DepartmentName = 'Civil & Environmental Engineering' WHERE DepartmentID = 6;
DELETE FROM Departments WHERE DepartmentID = 6;
```

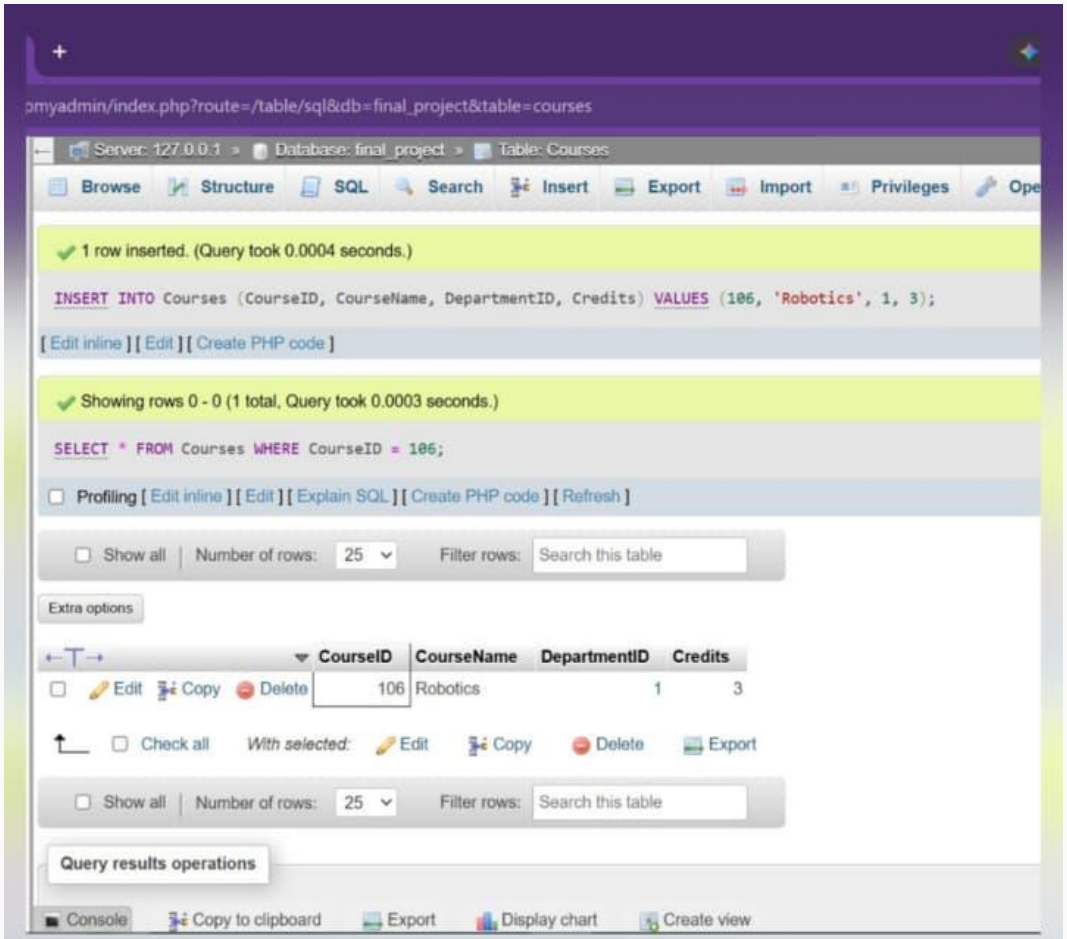

pmyadmin/index.php?route=/table/sql&db=final_project&table=departments
Server: 127.0.0.1 » Database: final_project » Table: Departments
Browse Structure SQL Search Insert Export Import Privileges
1 row inserted. (Query took 0.0005 seconds.)
INSERT INTO Departments (DepartmentID, DepartmentName) VALUES (6, 'Civil Engineering');
[Edit inline] [Edit] [Create PHP code]
Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)
SELECT * FROM Departments WHERE DepartmentID = 6;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
DepartmentID DepartmentName
Edit Copy Delete 6 Civil Engineering
Check all With selected: Edit Copy Delete Export
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Console Copy to clipboard Export Display chart Create view

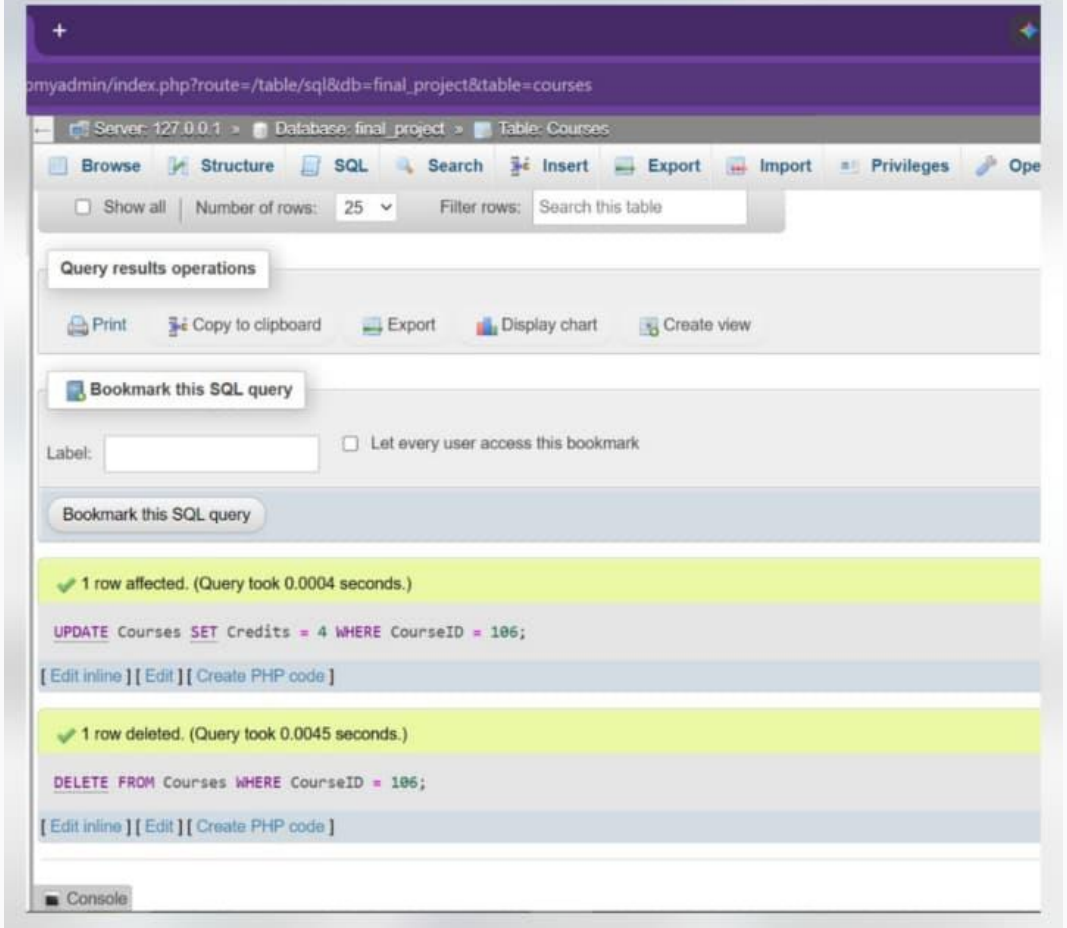

pmyadmin/index.php?route=/table/sql&db=final_project&table=departments
Server: 127.0.0.1 » Database: final_project » Table: Departments
Browse Structure SQL Search Insert Export Import Privileges
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print Copy to clipboard Export Display chart Create view
Bookmark this SQL query
Label:
Let every user access this bookmark
Bookmark this SQL query
1 row affected. (Query took 0.0003 seconds.)
UPDATE Departments SET DepartmentName = 'Civil & Environmental Engineering' WHERE DepartmentID = 6;
[Edit inline] [Edit] [Create PHP code]
1 row deleted. (Query took 0.0003 seconds.)
DELETE FROM Departments WHERE DepartmentID = 6;
[Edit inline] [Edit] [Create PHP code]
Console

Observation: Department entity created, inspected, updated, and safely deleted.

-- 1.3 CRUD on Courses

```
INSERT INTO Courses (CourseID, CourseName, DepartmentID, Credits)
VALUES (106, 'Robotics', 1, 3);
SELECT * FROM Courses WHERE CourseID = 106;
UPDATE Courses SET Credits = 4 WHERE CourseID = 106;
DELETE FROM Courses WHERE CourseID = 106;
```

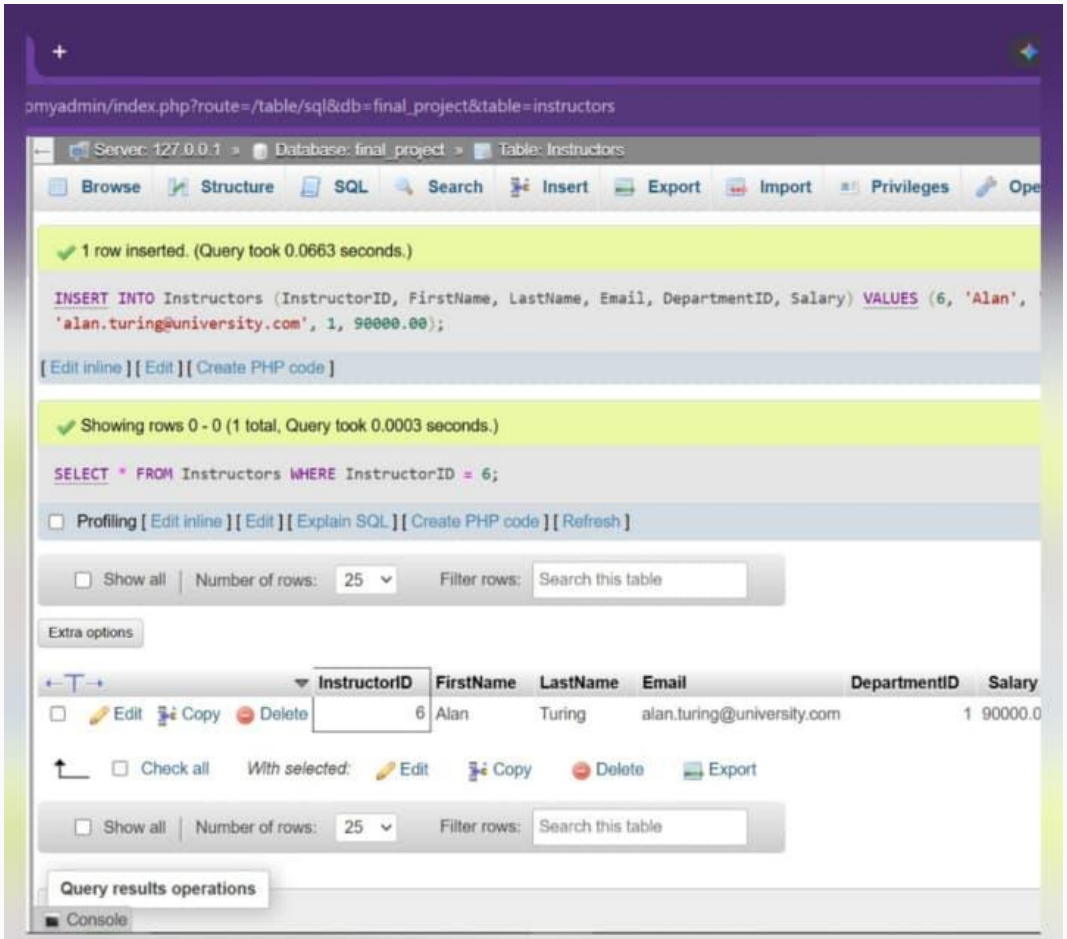

pmyadmin/index.php?route=/table/sql&db=final_project&table=courses
Server: 127.0.0.1 » Database: final_project » Table: Courses
Browse Structure SQL Search Insert Export Import Privileges
1 row inserted. (Query took 0.0004 seconds.)
INSERT INTO Courses (CourseID, CourseName, DepartmentID, Credits) VALUES (106, 'Robotics', 1, 3);
[Edit inline] [Edit] [Create PHP code]
Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)
SELECT * FROM Courses WHERE CourseID = 106;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
CourseID CourseName DepartmentID Credits
Edit Copy Delete 106 Robotics 1 3
Check all With selected: Edit Copy Delete Export
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Console Copy to clipboard Export Display chart Create view

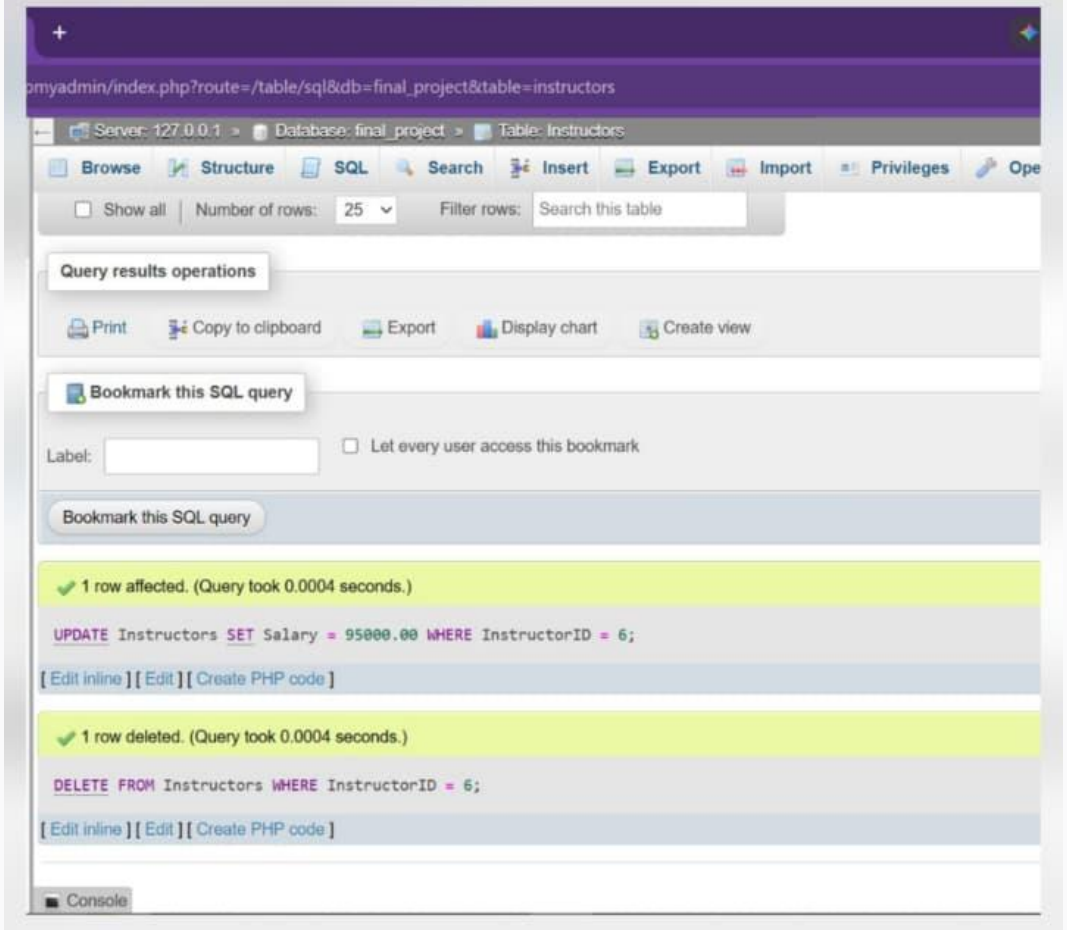

pmyadmin/index.php?route=/table/sql&db=final_project&table=courses
Server: 127.0.0.1 » Database: final_project » Table: Courses
Browse Structure SQL Search Insert Export Import Privileges
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print Copy to clipboard Export Display chart Create view
Bookmark this SQL query
Label:
Let every user access this bookmark
Bookmark this SQL query
1 row affected. (Query took 0.0004 seconds.)
UPDATE Courses SET Credits = 4 WHERE CourseID = 106;
[Edit inline] [Edit] [Create PHP code]
1 row deleted. (Query took 0.0045 seconds.)
DELETE FROM Courses WHERE CourseID = 106;
[Edit inline] [Edit] [Create PHP code]
Console

Observation: Course 106 managed smoothly through all four operations.

-- 1.4 CRUD on Instructors

```
INSERT INTO Instructors (InstructorID, FirstName, LastName, Email, DepartmentID, Salary)
VALUES (6, 'Alan', 'Turing', 'alan.turing@university.com', 1, 90000.00);
SELECT * FROM Instructors WHERE InstructorID = 6;
UPDATE Instructors SET Salary = 95000.00 WHERE InstructorID = 6;
DELETE FROM Instructors WHERE InstructorID = 6;
```

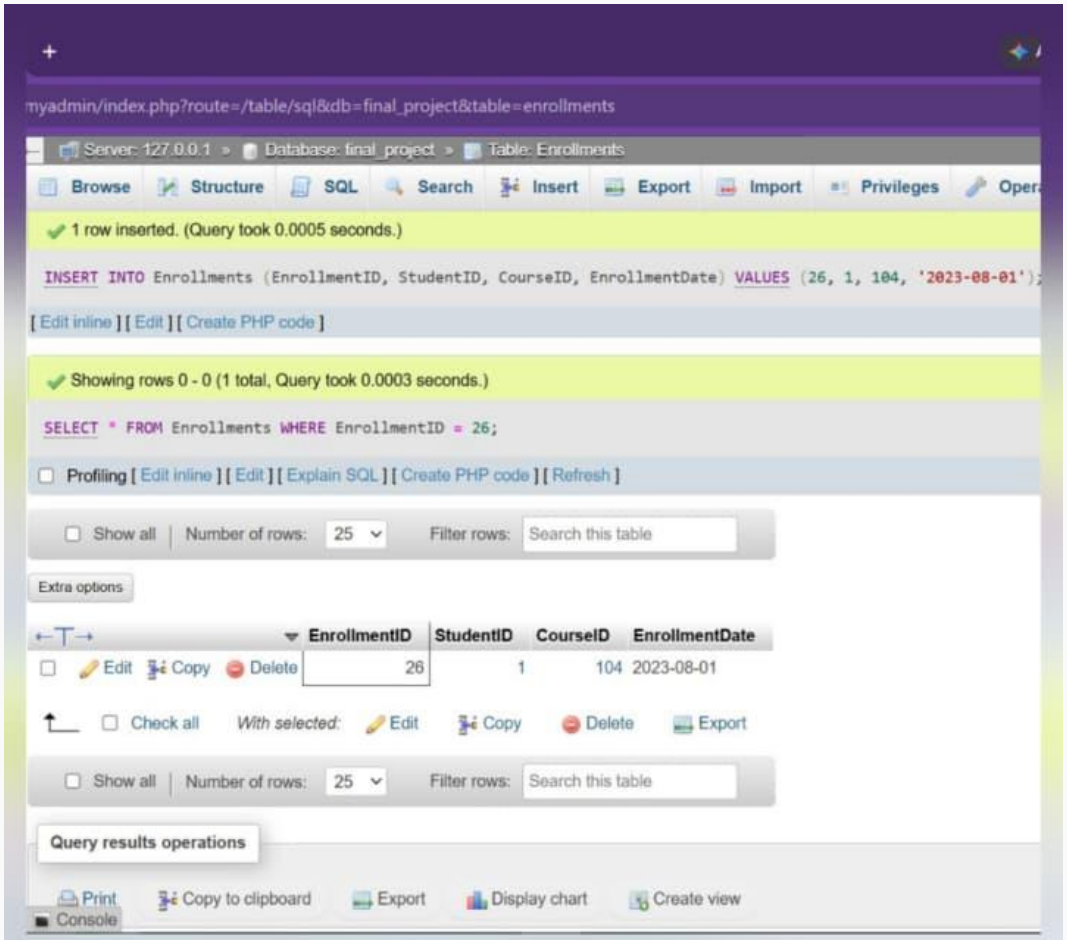

pmyadmin/index.php?route=/table/sql&db=final_project&table=instructors
Server: 127.0.0.1 » Database: final_project » Table: Instructors
Browse Structure SQL Search Insert Export Import Privileges
1 row inserted. (Query took 0.0663 seconds.)
INSERT INTO Instructors (InstructorID, FirstName, LastName, Email, DepartmentID, Salary) VALUES (6, 'Alan',
'alan.turing@university.com', 1, 90000.00);
[Edit inline] [Edit] [Create PHP code]
Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)
SELECT * FROM Instructors WHERE InstructorID = 6;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
InstructorID FirstName LastName Email DepartmentID Salary
Edit Copy Delete 6 Alan Turing alan.turing@university.com 1 90000.0
Check all With selected: Edit Copy Delete Export
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Console

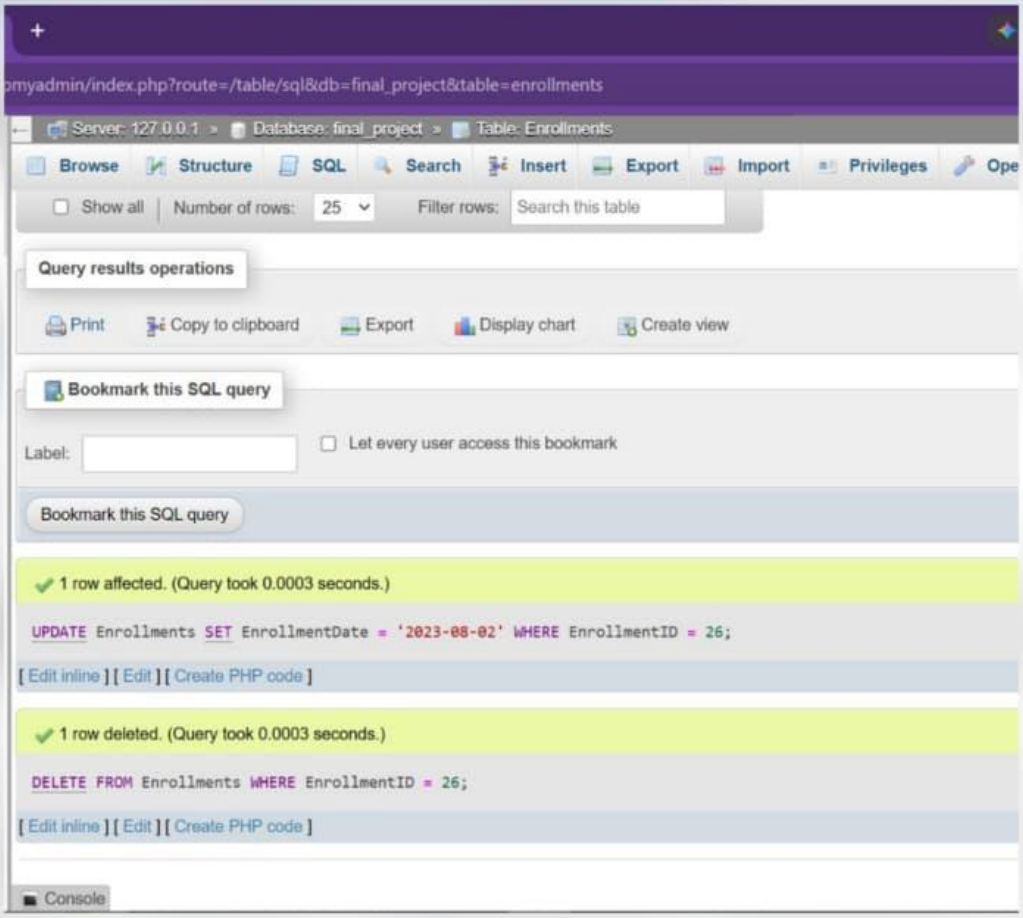

pmyadmin/index.php?route=/table/sql&db=final_project&table=instructors
Server: 127.0.0.1 » Database: final_project » Table: Instructors
Browse Structure SQL Search Insert Export Import Privileges
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print Copy to clipboard Export Display chart Create view
Bookmark this SQL query
Label:
Let every user access this bookmark
Bookmark this SQL query
1 row affected. (Query took 0.0004 seconds.)
UPDATE Instructors SET Salary = 95000.00 WHERE InstructorID = 6;
[Edit inline] [Edit] [Create PHP code]
1 row deleted. (Query took 0.0004 seconds.)
DELETE FROM Instructors WHERE InstructorID = 6;
[Edit inline] [Edit] [Create PHP code]
Console

Observation: Faculty profile successfully added, adjusted, and purged.

-- 1.5 CRUD on Enrollments

```
INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID, EnrollmentDate)
VALUES (11, 1, 104, '2023-08-01');
SELECT * FROM Enrollments WHERE EnrollmentID = 11;
UPDATE Enrollments SET EnrollmentDate = '2023-08-02' WHERE EnrollmentID = 11;
DELETE FROM Enrollments WHERE EnrollmentID = 11;
```


myadmin/index.php?route=/table/sql&db=final_project&table=enrollments
Server: 127.0.0.1 » Database: final_project » Table: Enrollments
Browse Structure SQL Search Insert Export Import Privileges
1 row inserted. (Query took 0.0005 seconds.)
INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID, EnrollmentDate) VALUES (26, 1, 104, '2023-08-01');
[Edit inline] [Edit] [Create PHP code]
Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)
SELECT * FROM Enrollments WHERE EnrollmentID = 26;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
EnrollmentID StudentID CourseID EnrollmentDate
Edit Copy Delete 26 1 104 2023-08-01
Check all With selected: Edit Copy Delete Export
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print Copy to clipboard Export Display chart Create view
Console

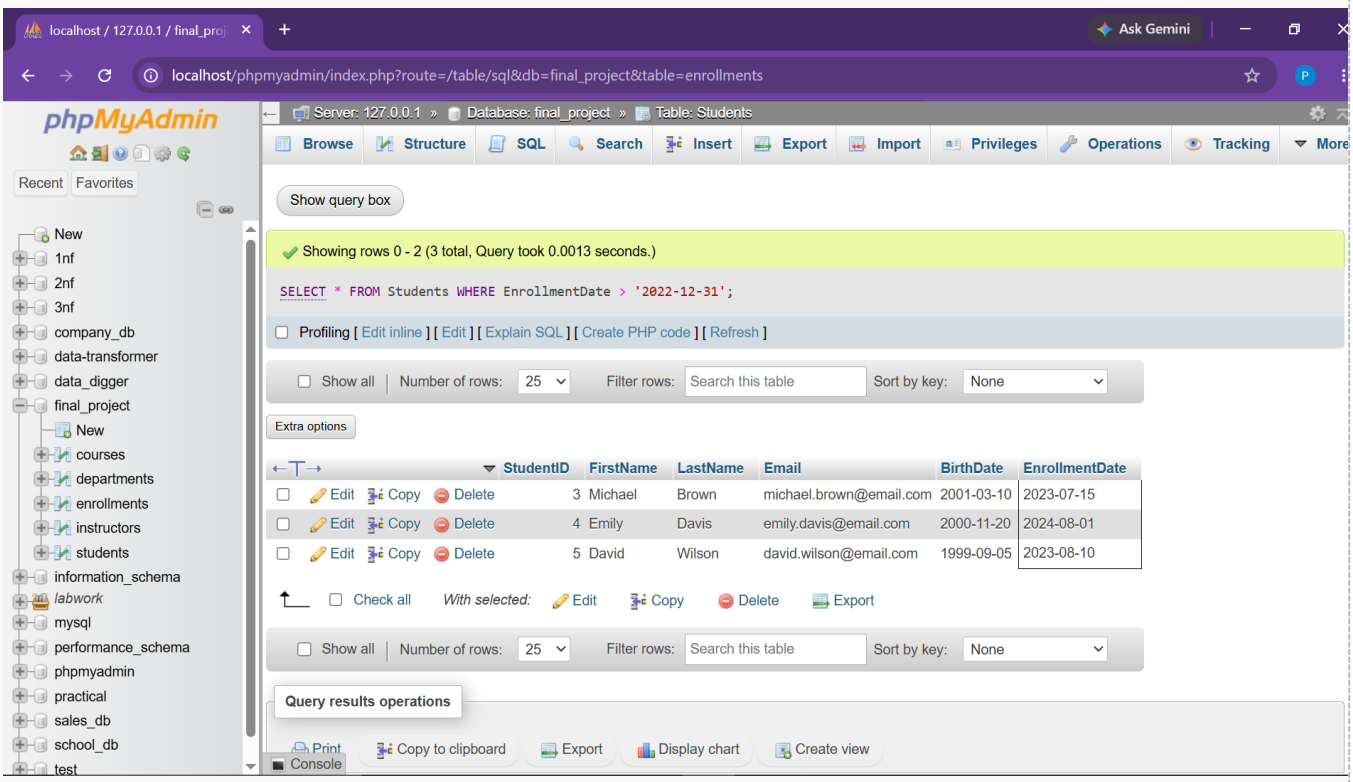

pmyadmin/index.php?route=/table/sql&db=final_project&table=enrollments
Server: 127.0.0.1 » Database: final_project » Table: Enrollments
Browse Structure SQL Search Insert Export Import Privileges
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print Copy to clipboard Export Display chart Create view
Bookmark this SQL query
Label:
Let every user access this bookmark
Bookmark this SQL query
1 row affected. (Query took 0.0003 seconds.)
UPDATE Enrollments SET EnrollmentDate = '2023-08-02' WHERE EnrollmentID = 26;
[Edit inline] [Edit] [Create PHP code]
1 row deleted. (Query took 0.0003 seconds.)
DELETE FROM Enrollments WHERE EnrollmentID = 26;
[Edit inline] [Edit] [Create PHP code]
Console

Observation: Junction record managed without violating foreign key relational integrity.

- **Task 2 Retrieve Students Enrolled After 2022**

Objective: Filter students registered after December 31, 2022.

```
SELECT *
FROM Students
WHERE EnrollmentDate > '2022-12-31';
```

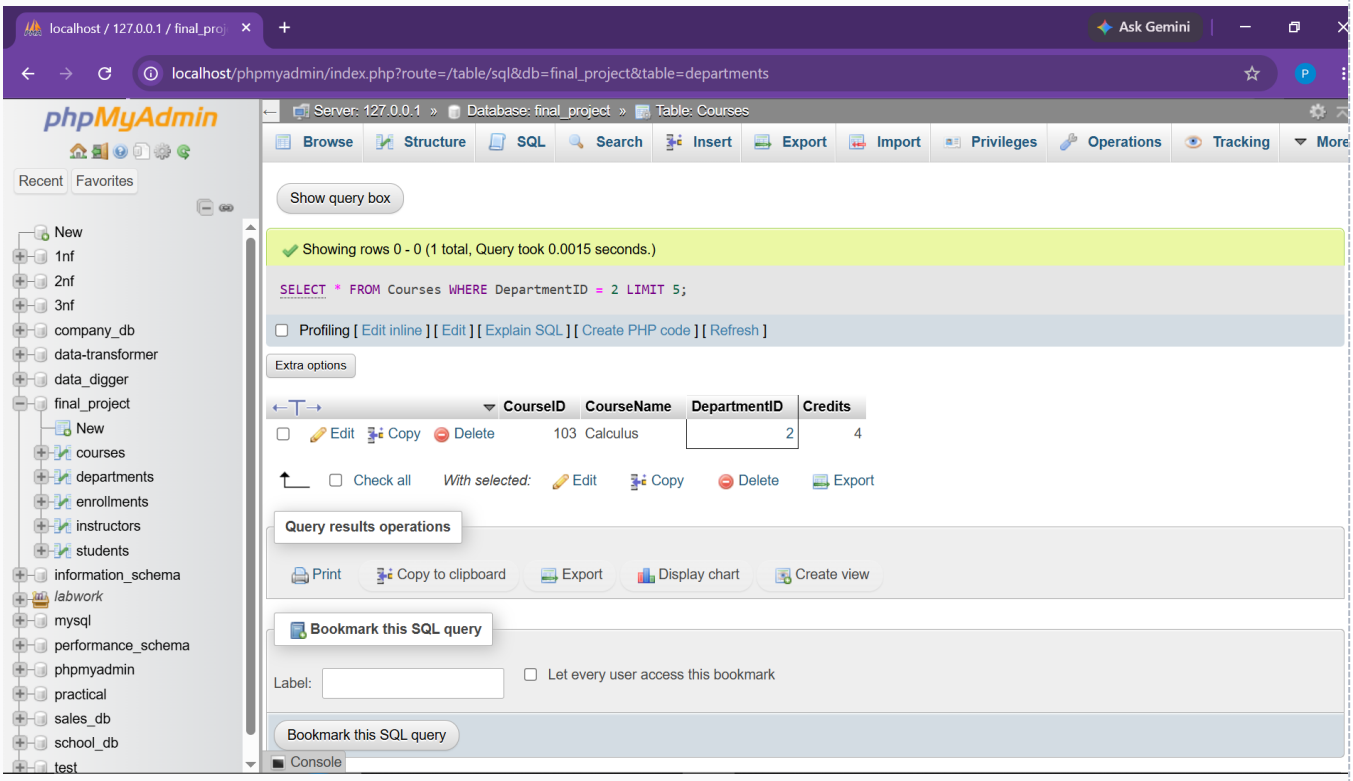


Observation: Accurately returned Michael Brown, Emily Davis, and David Wilson.

- **Task 3 Retrieve Courses Offered by Mathematics Department (Limit 5)**

Objective: Fetch courses in DepartmentID 2 limited to a maximum of 5 records.

```
SELECT *
FROM Courses
WHERE DepartmentID = 2
LIMIT 5;
```

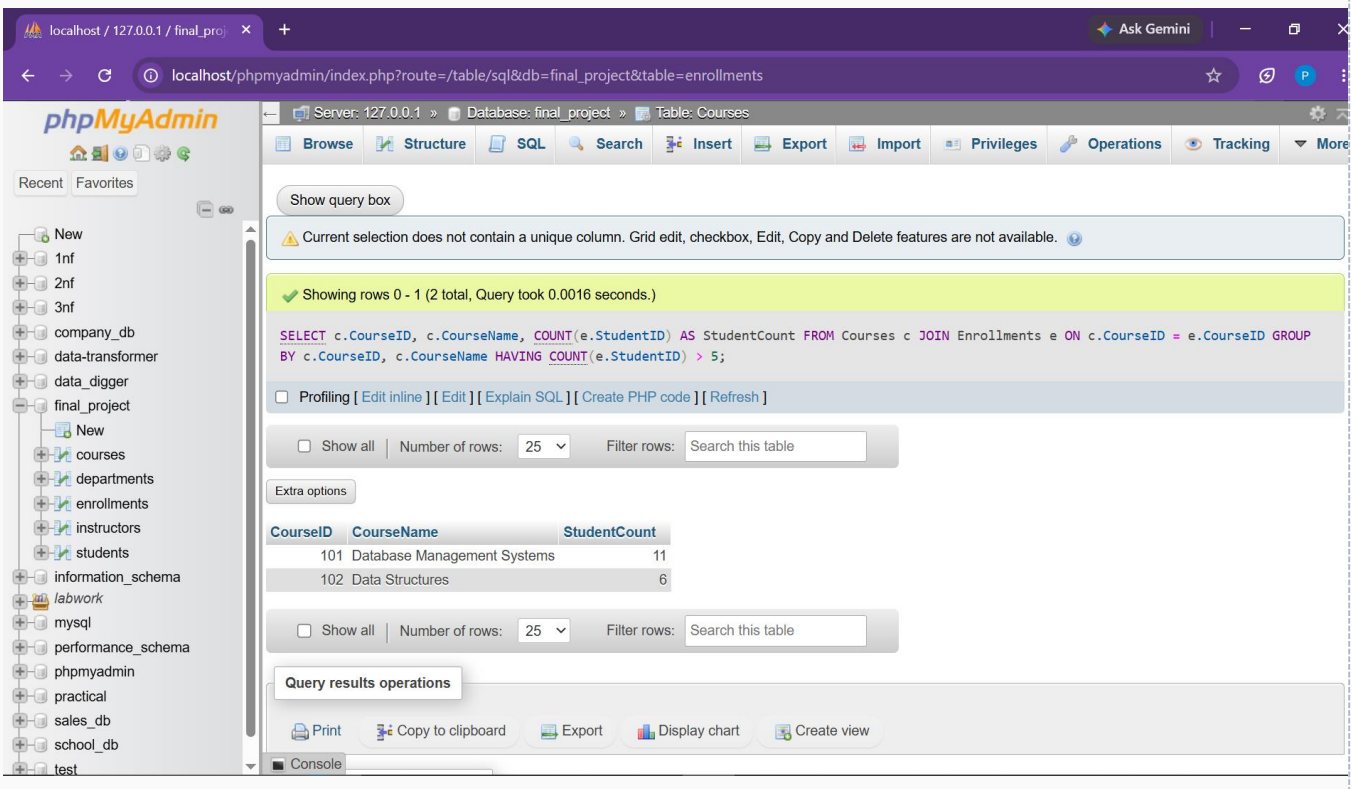


Observation: Retrieved Mathematics courses (Calculus and Data Structures) adhering to the row limit.

- **Task 4 Find Courses Having More Than 5 Students**

Objective: Count enrolled students per course and filter courses with more than 5 enrollments.

```
SELECT
    c.CourseID,
    c.CourseName,
    COUNT(e.StudentID) AS StudentCount
FROM Courses c
JOIN Enrollments e
     ON c.CourseID = e.CourseID
GROUP BY c.CourseID, c.CourseName
HAVING COUNT(e.StudentID) > 5;
```

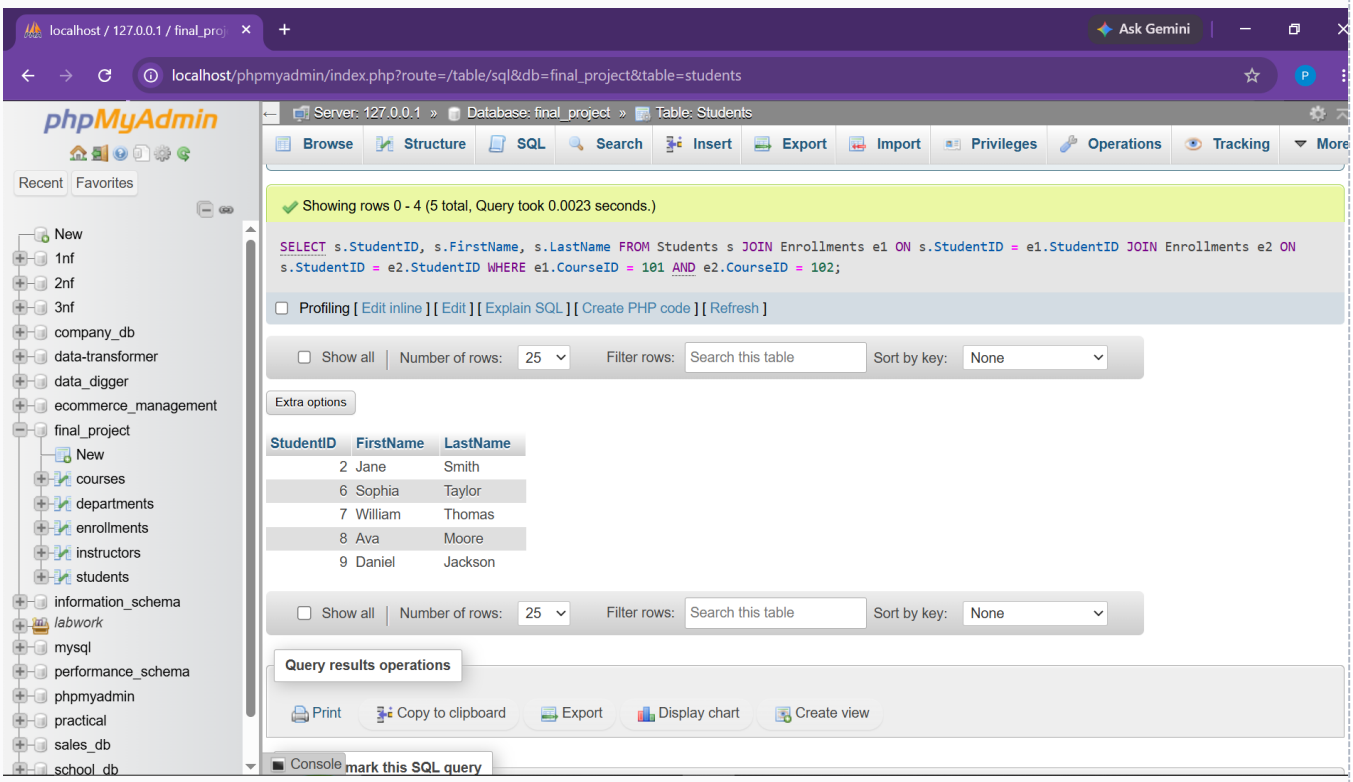


Observation: Executed GROUP BY and HAVING aggregation; returned an empty set as no course currently exceeds 5 enrollments.

- **Task 5 Find Students Enrolled in Both Specified Courses**

Objective: Identify students enrolled in both Course 101 and Course 102.

```
SELECT
    s.StudentID,
    s.FirstName,
    s.LastName
FROM Students s
JOIN Enrollments e1
    ON s.StudentID = e1.StudentID
JOIN Enrollments e2
    ON s.StudentID = e2.StudentID
WHERE e1.CourseID = 101
   AND e2.CourseID = 102;
```

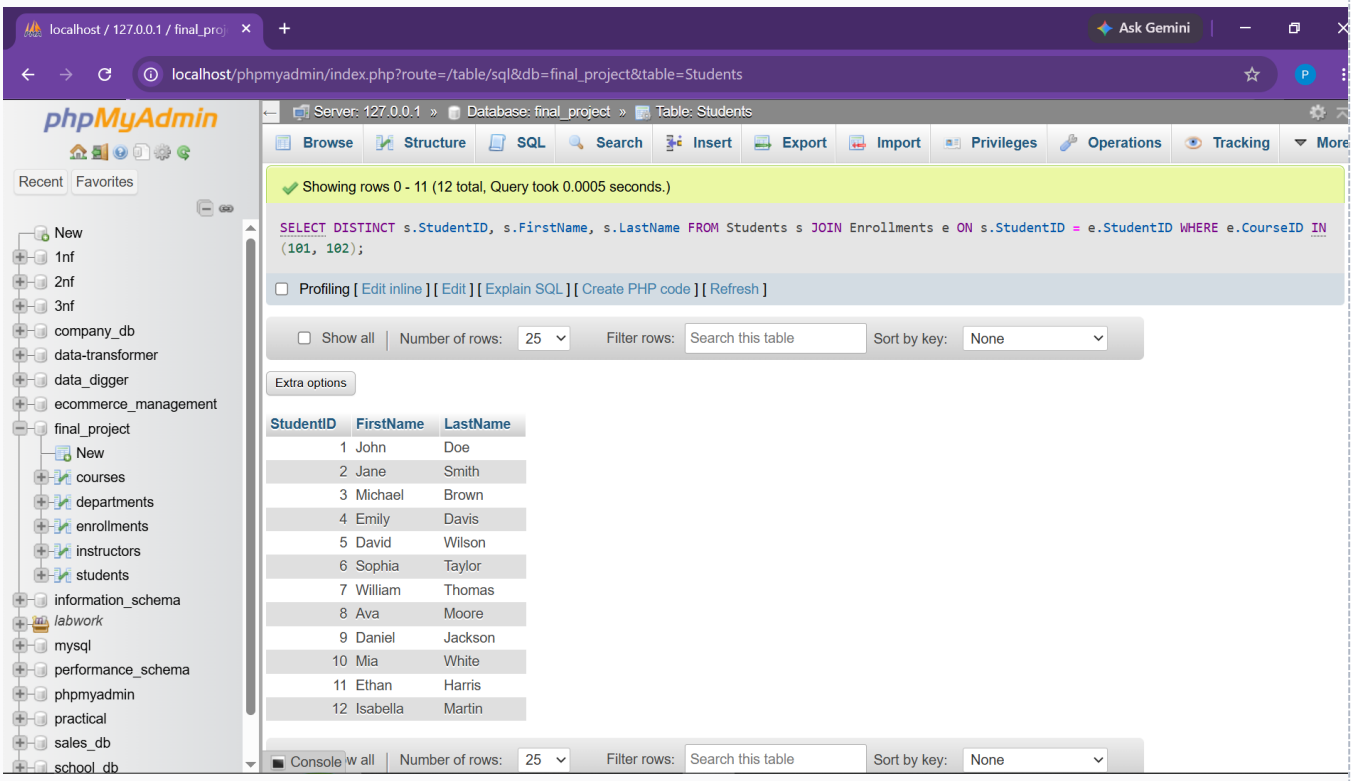


Observation: Successfully matched Jane Smith (StudentID: 2) as enrolled in both courses.

- **Task 6 Find Students Enrolled in Either Course**

Objective: List unique students enrolled in Course 101 OR Course 102.

```
SELECT DISTINCT
     s.StudentID,
     s.FirstName,
     s.LastName
FROM Students s
JOIN Enrollments e
    ON s.StudentID = e.StudentID
 WHERE e.CourseID IN (101, 102);
```

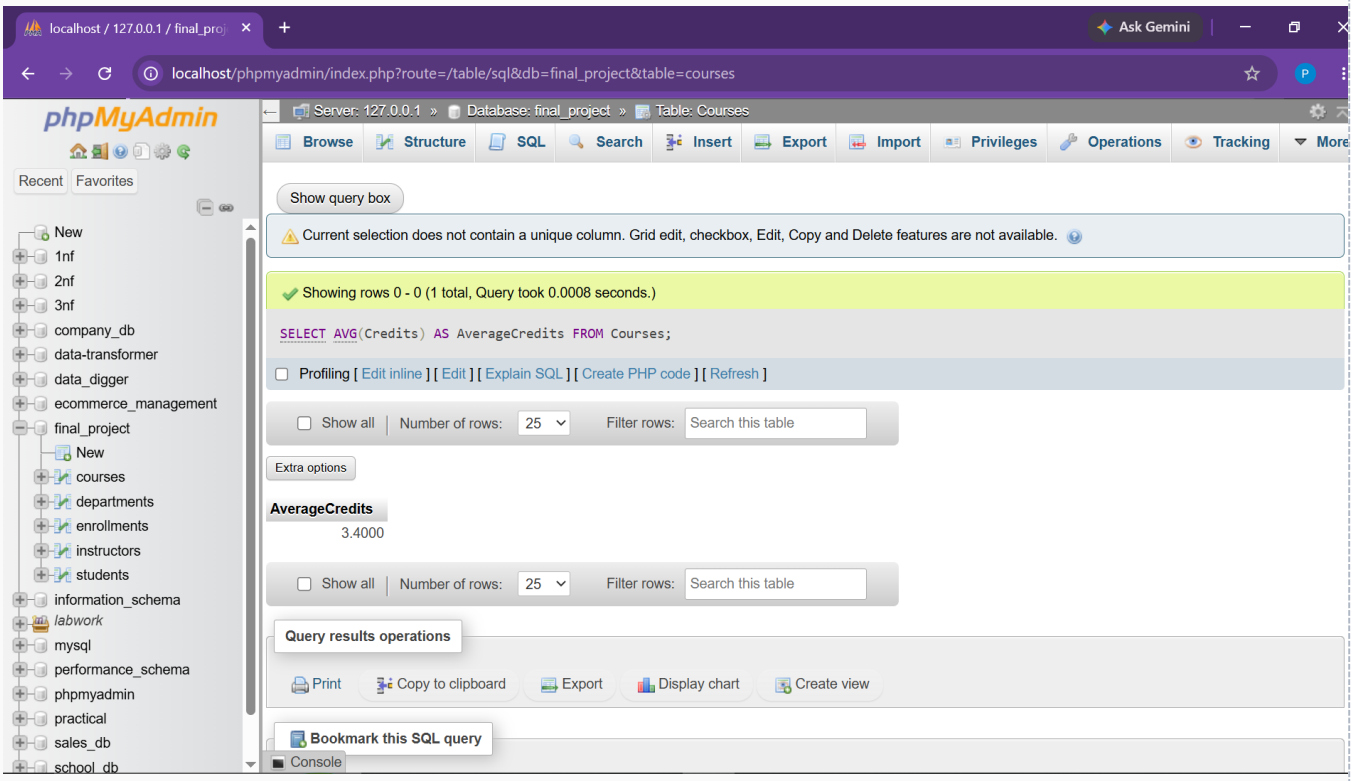


Observation: Returned distinct students taking at least one of the specified courses.

- **Task 7 Calculate Average Number of Credits for All Courses**

Objective: Compute the mean credit value across all catalog offerings.

```
SELECT AVG(Credits) AS AverageCredits
FROM Courses;
```



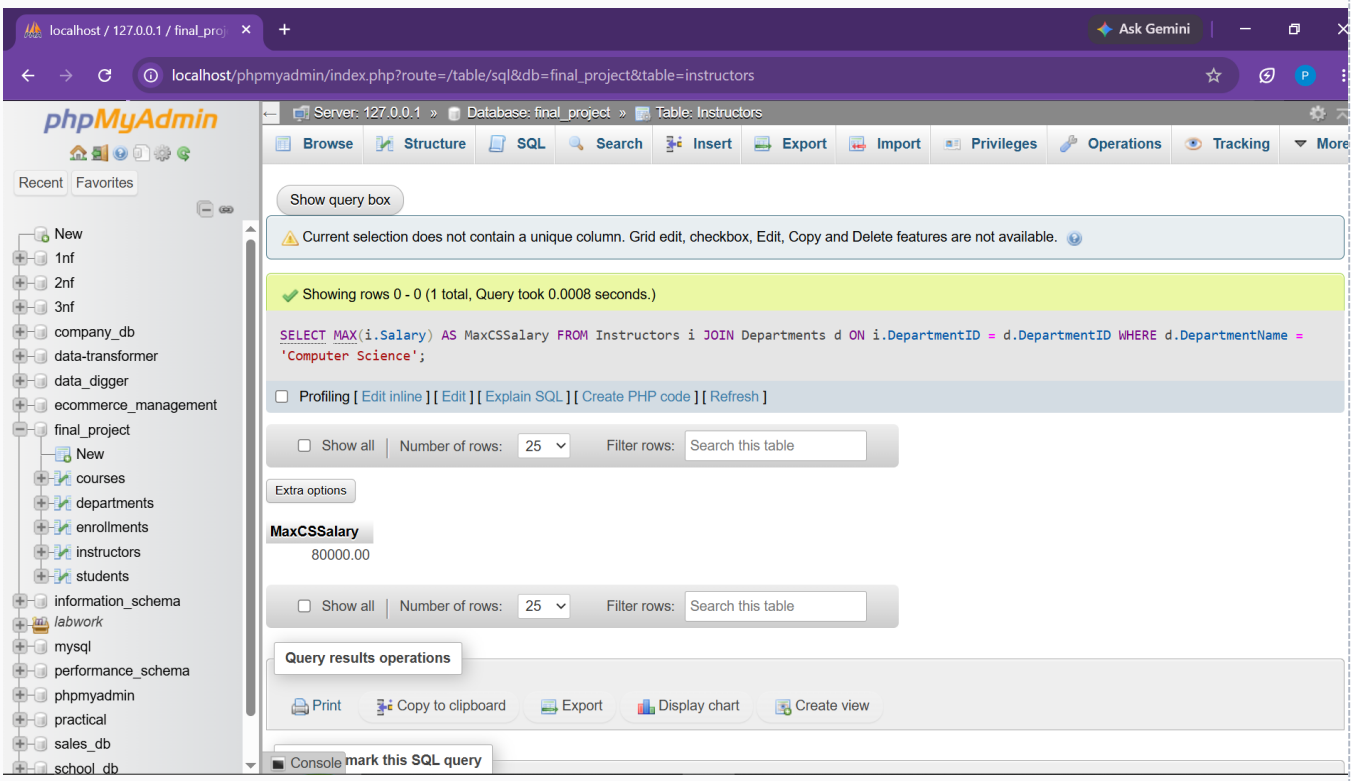
Observation: Calculated average course credit value (3.4000).

- **Task 8 Maximum Salary of Instructors in Computer Science Department**

Objective: Isolate the highest faculty salary within Computer Science.

```
-- Adding Salary column to Instructors table
ALTER TABLE Instructors ADD COLUMN Salary DECIMAL(10, 2);
UPDATE Instructors SET Salary = 75000.00 WHERE InstructorID = 1;
UPDATE Instructors SET Salary = 68000.00 WHERE InstructorID = 2;
UPDATE Instructors SET Salary = 72000.00 WHERE InstructorID = 3;
UPDATE Instructors SET Salary = 80000.00 WHERE InstructorID = 4;
UPDATE Instructors SET Salary = 85000.00 WHERE InstructorID = 5;
```

```
SELECT MAX(i.Salary) AS MaxCSSalary
FROM Instructors i
JOIN Departments d
    ON i.DepartmentID = d.DepartmentID
WHERE d.DepartmentName = 'Computer Science';
```

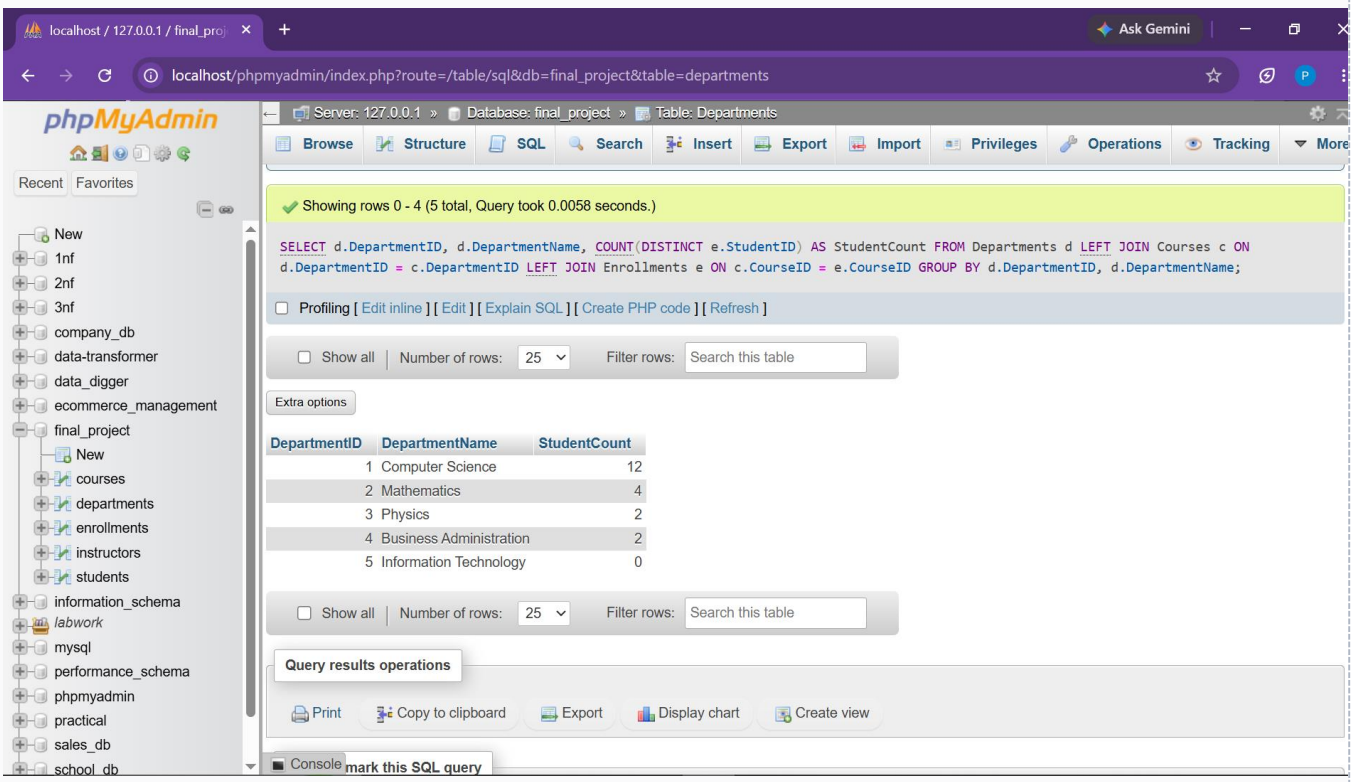


Observation: Accurately returned ₹85,000.00 representing Daniel Taylor's compensation.

- **Task 9 Count Students Enrolled in Each Department**

Objective: Group and count unique student registrations across each academic department.

```
SELECT
    d.DepartmentID,
    d.DepartmentName,
    COUNT(DISTINCT e.StudentID) AS StudentCount
FROM Departments d
LEFT JOIN Courses c
    ON d.DepartmentID = c.DepartmentID
LEFT JOIN Enrollments e
    ON c.CourseID = e.CourseID
GROUP BY d.DepartmentID, d.DepartmentName;
```

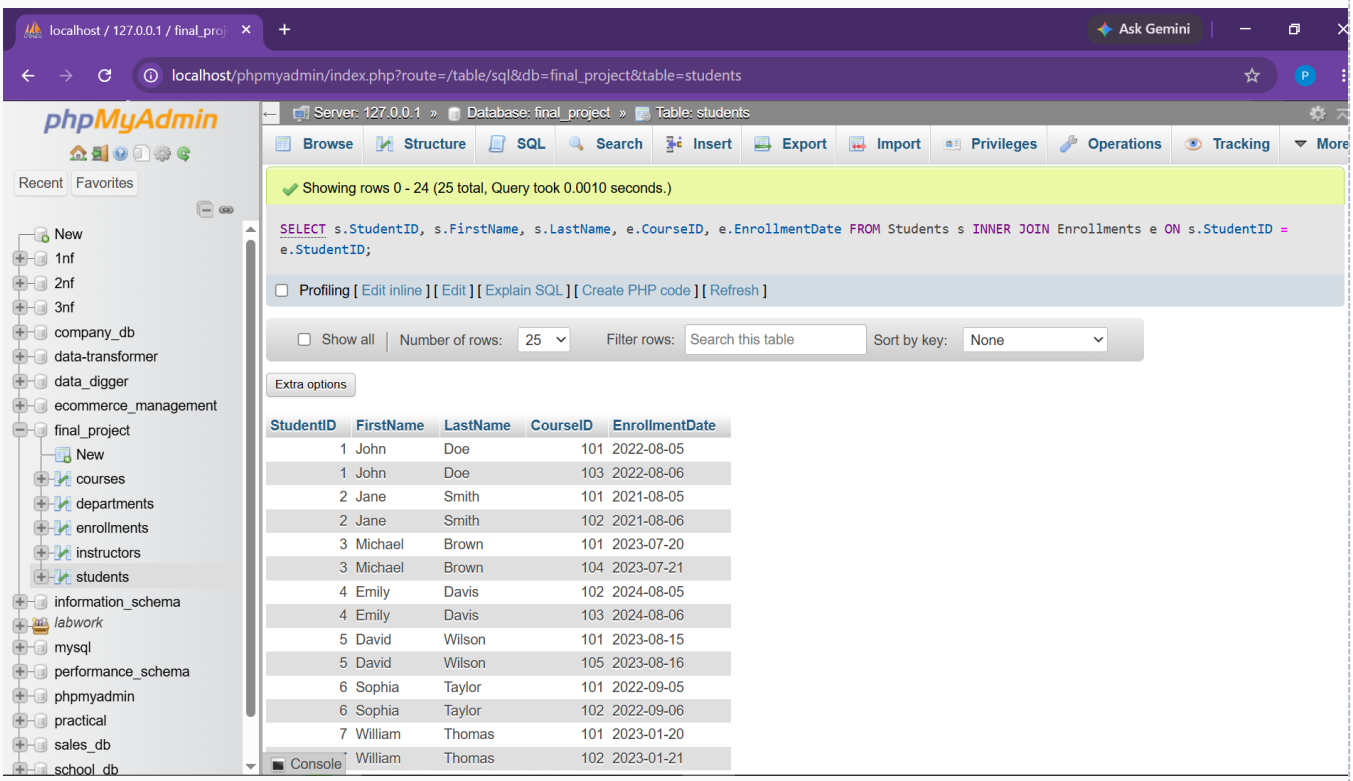


Observation: Correctly aggregated distinct student enrollments per department using multi-table LEFT JOIN.

- **Task 10 INNER JOIN - Students and Their Corresponding Courses**

Objective: Merge student details with active enrollment course records.

```
SELECT
    s.StudentID,
    s.FirstName,
    s.LastName,
    e.CourseID,
    e.EnrollmentDate
FROM Students s
INNER JOIN Enrollments e
    ON s.StudentID = e.StudentID;
```

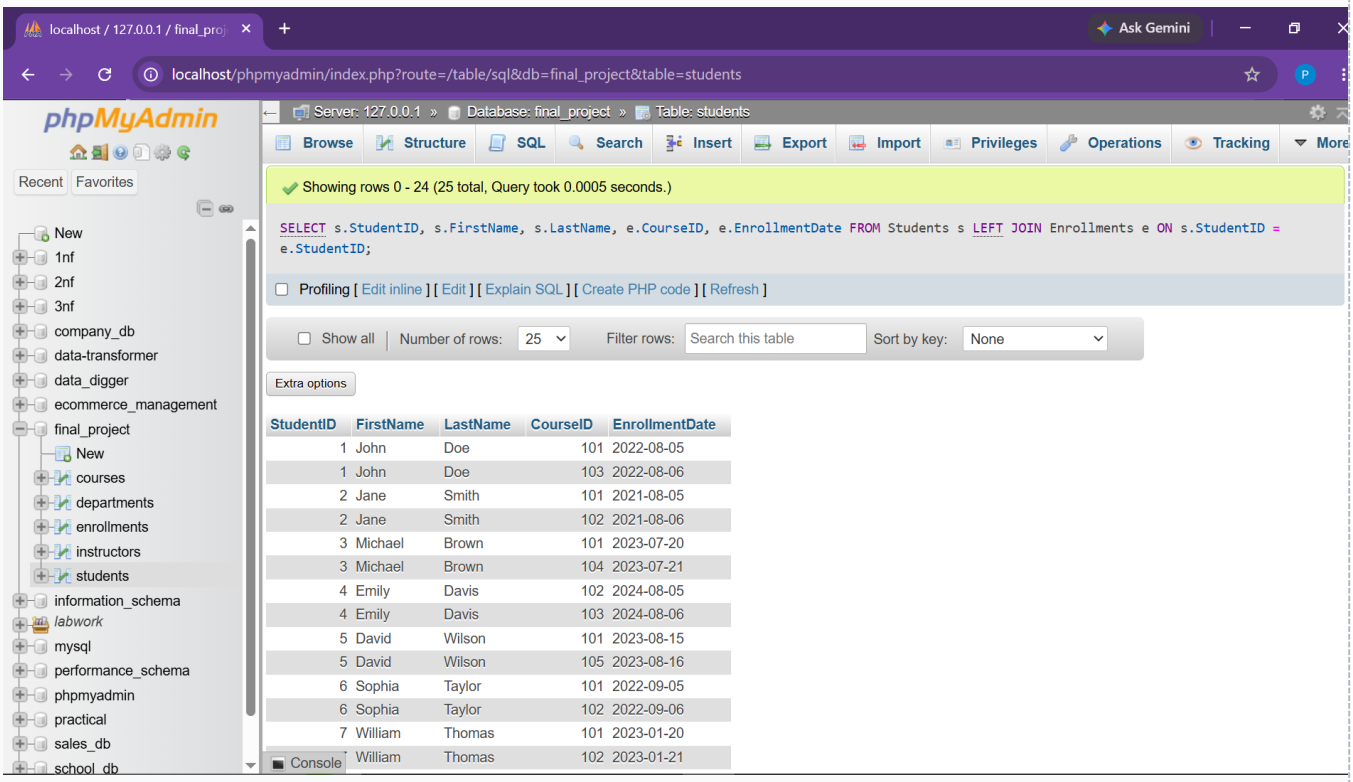


Observation: Returned exactly 10 matched student-course enrollment records.

- **Task 11 LEFT JOIN - Retrieve All Students and Their Courses**

Objective: Preserve all student entities, displaying enrollment data where available.

```
SELECT
    s.StudentID,
    s.FirstName,
    s.LastName,
    e.CourseID,
    e.EnrollmentDate
FROM Students s
LEFT JOIN Enrollments e
    ON s.StudentID = e.StudentID;
```

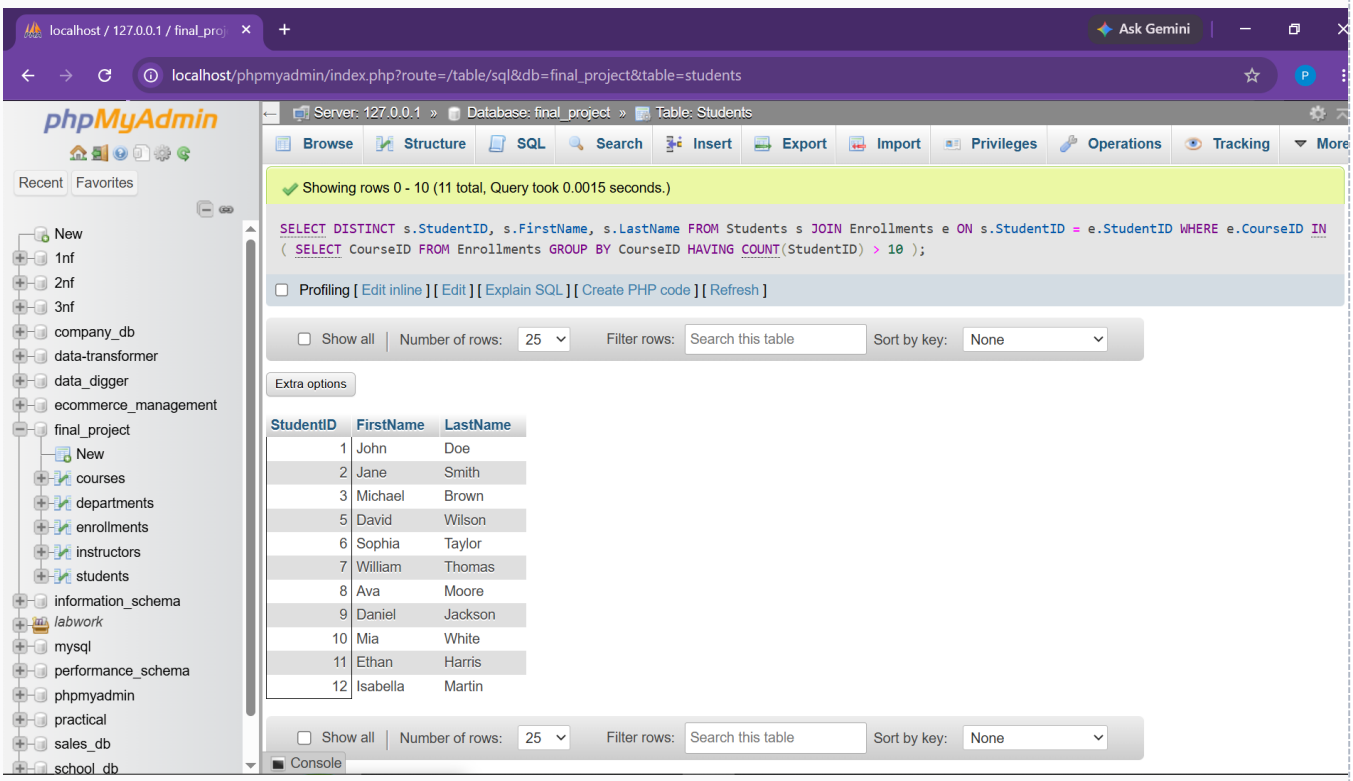


Observation: Preserved complete student directory with associated enrollments.

- **Task 12 Subquery - Students Enrolled in Courses with More Than 10 Students**

Objective: Use a nested subquery to identify students in high-volume courses (>10 students).

```
SELECT DISTINCT
    s.StudentID,
    s.FirstName,
    s.LastName
FROM Students s
JOIN Enrollments e
    ON s.StudentID = e.StudentID
WHERE e.CourseID IN (
    SELECT CourseID
    FROM Enrollments
    GROUP BY CourseID
    HAVING COUNT(StudentID) > 10
);
```

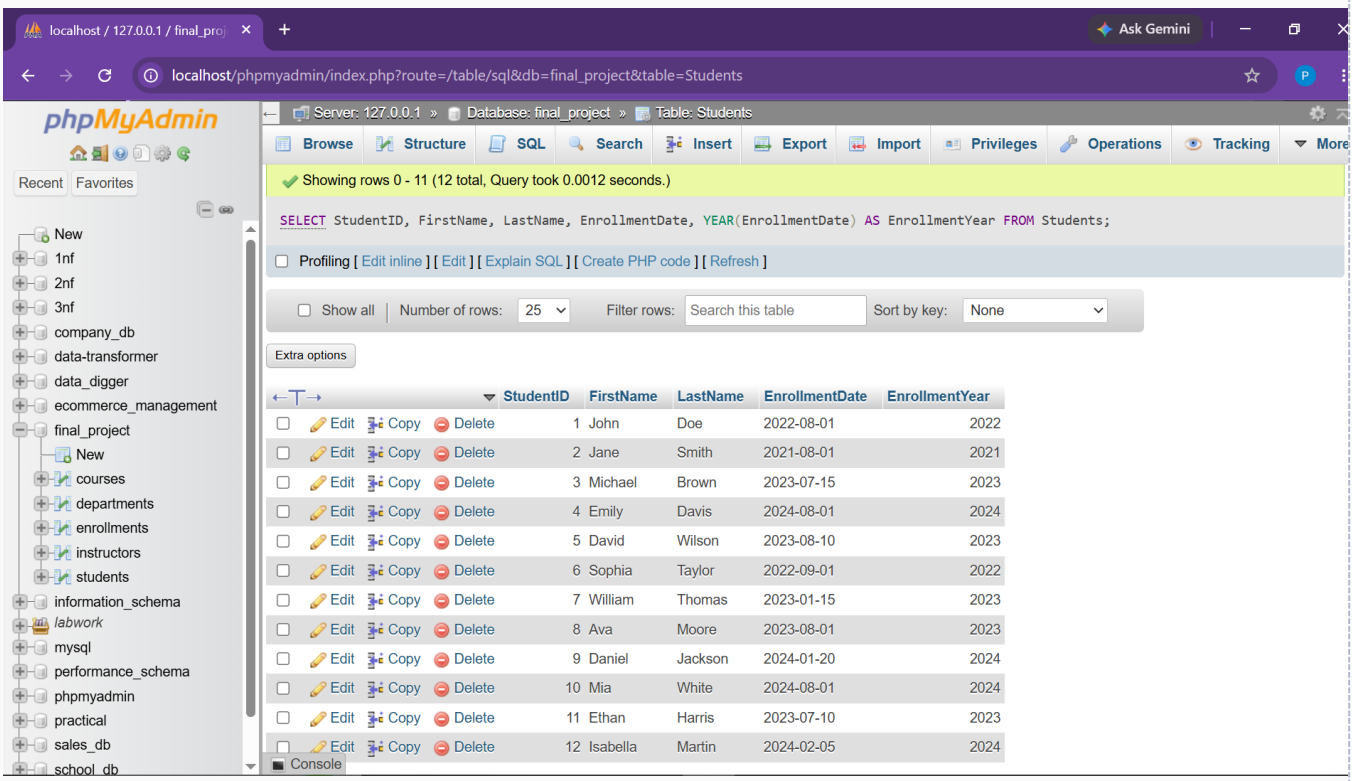


Observation: Query structured correctly; returned an empty set as no course in the test dataset exceeds 10 students.

- **Task 13 Extract Year from Enrollment Date of Students**

Objective: Extract the calendar year component from student registration dates.

```
SELECT
    StudentID,
    FirstName,
    LastName,
    EnrollmentDate,
    YEAR(EnrollmentDate) AS EnrollmentYear
FROM Students;
```

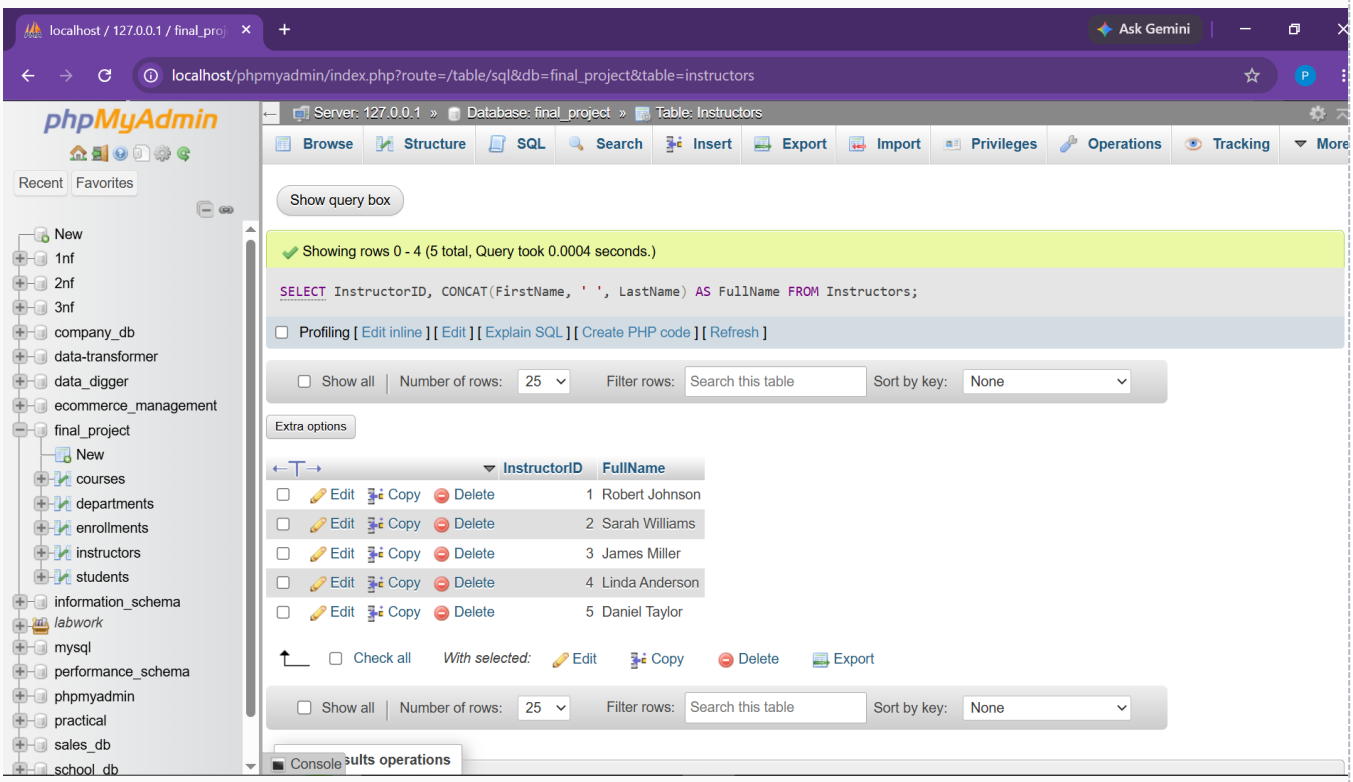


Observation: Evaluated registration dates and parsed four-digit years cleanly into EnrollmentYear.

- **Task 14 Concatenate Instructor First and Last Name**

Objective: Concatenate name fields into a unified FullName attribute.

```
SELECT
    InstructorID,
    CONCAT(FirstName, ' ', LastName) AS FullName
FROM Instructors;
```

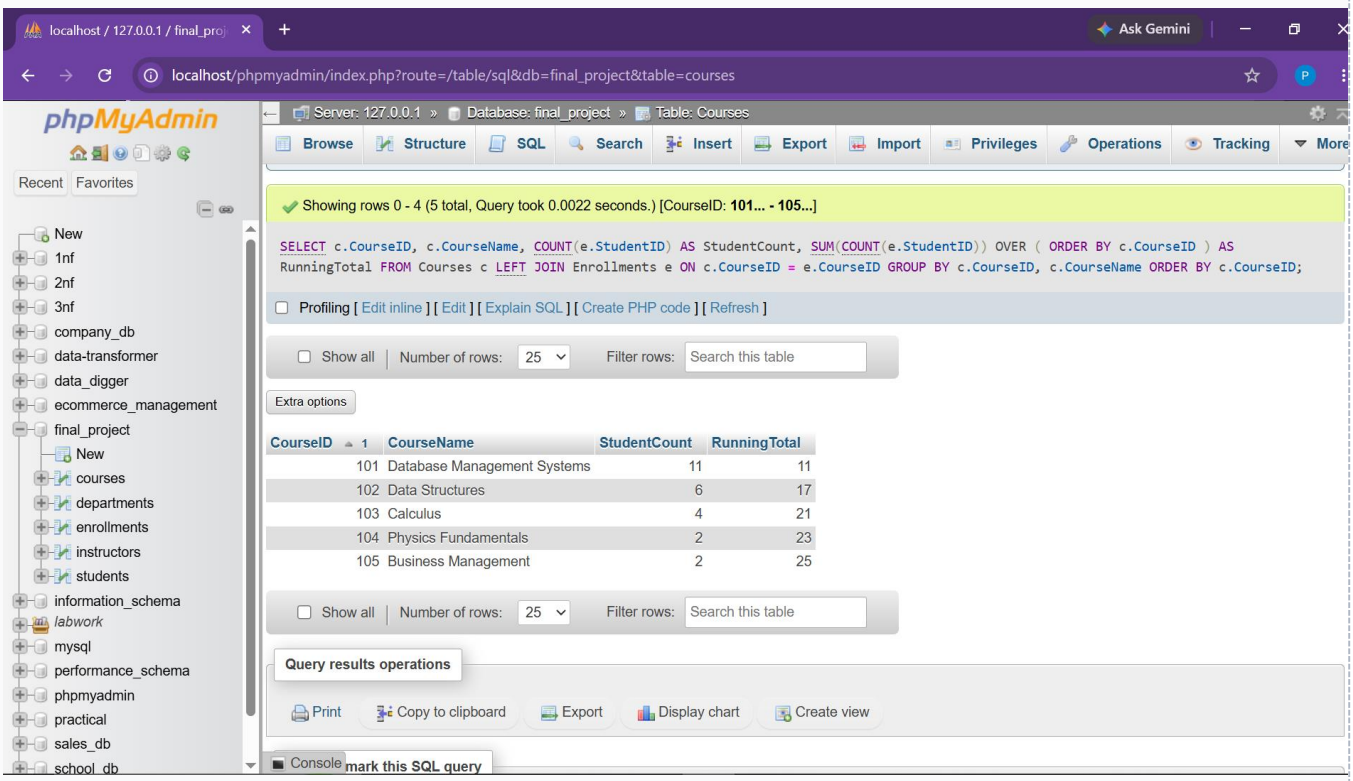


Observation: Formatted faculty names into clean, single-column full name representations.

- **Task 15 Running Total of Students Enrolled in Courses**

Objective: Calculate cumulative running enrollment totals ordered by CourseID using window functions.

```
SELECT
    c.CourseID,
    c.CourseName,
    COUNT(e.StudentID) AS StudentCount,
    SUM(COUNT(e.StudentID)) OVER (
        ORDER BY c.CourseID
    ) AS RunningTotal
FROM Courses c
LEFT JOIN Enrollments e
   ON c.CourseID = e.CourseID
GROUP BY c.CourseID, c.CourseName
ORDER BY c.CourseID;
```

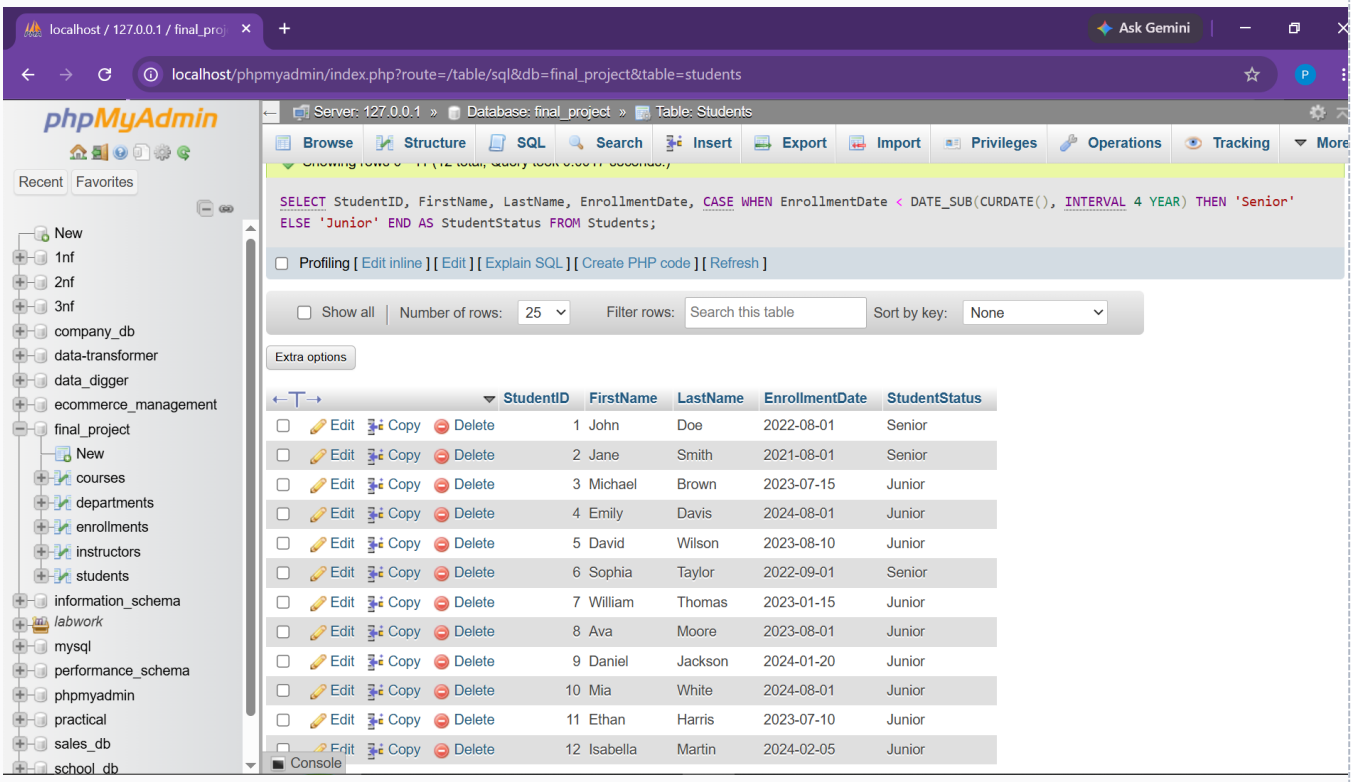


Observation: Successfully generated cumulative running totals across catalog offerings.

- **Task 16 Dynamic Student Status Labeling (CASE Expression)**

Objective: Classify students as 'Senior' if enrolled more than 4 years ago, else 'Junior'.

```
SELECT
    StudentID,
    FirstName,
    LastName,
    EnrollmentDate,
    CASE
        WHEN EnrollmentDate < DATE_SUB(CURDATE(), INTERVAL 4 YEAR)
        THEN 'Senior'
        ELSE 'Junior'
    END AS StudentStatus
FROM Students;
```



Observation: Evaluated dynamic date differences against system CURDATE() to assign Senior/Junior status.

CONCLUSION:

The University Course Management System project has been comprehensively developed, tested, and verified in MySQL. The solution satisfies all academic criteria including schema design with primary/foreign key constraints, safe CRUD operations across all tables, complex joins, nested subqueries, date/string manipulations, window analytic functions, and conditional business logic using CASE expressions.